The Limits of Predictions for Online Bipartite Matching: A Unified Experimental Study

[Author Name] — Master's Thesis (draft)

Abstract

Learning-augmented (“with-predictions”) algorithms for online bipartite matching have proliferated. One family consumes a prediction of how contended each resource will be. Another tests a prediction of the arrival mix on a short prefix of the requests — a vanishing fraction of the instance — before committing to follow it or to fall back on an advice-free baseline. These algorithms have been analyzed in isolation. We give the first unified experimental study. We place all of them on one harness: a common structured prediction-error model, a common optimum, and confidence intervals. We run it on synthetic graphs, six real-world graphs, and real request traces. Our central finding is that on average-case inputs the value of predictions is robustness insurance, not a performance lever. The advice-free baseline is already near-optimal, so the consistency upside of good advice is small, whereas unguarded prediction-following can crash far below the baseline; the practical worth of the sophisticated algorithms is that they never crash. We close with a theoretical outlook — one proved consistency/robustness trade-off inequality, and a quantitative reading of our own experiments under which the prefix needed to decide whether to trust advice scales as the inverse square of the advice’s upside. The full formal development is separate work, and no theorem beyond the stated inequality is claimed here. Experiments and outlook deliver one message: on average-case matching, the advice’s upside is smaller than the price of finding out whether to trust it.

Contents

Chapter 1

Introduction

1.1 Background and motivation

Online bipartite matching is the canonical model of irrevocable sequential allocation. Resources are known in advance; requests arrive one at a time; each must be matched to an available compatible resource — or dropped — immediately, before the rest of the input is seen. It is the abstraction behind online advertising, ride-hailing dispatch, organ exchange, and request routing in modern serving systems. Because decisions cannot be undone, the difficulty lies entirely in committing well under uncertainty about the future.

A recent and influential idea is to give such an algorithm a **prediction** about the input — a hint about which resources will be contended, or how many requests of each kind will arrive — and to ask for two guarantees at once: **consistency**, meaning near-optimal performance when the prediction is good, and **robustness**, meaning no worse than a prediction-free algorithm when the prediction is wrong. For online matching specifically, two families of such *learning-augmented* algorithms have appeared: MinPredictedDegree, which consumes a per-resource degree prediction, and a family of *test-and-fallback* schemes, which test a request-type prediction on a short prefix of arrivals before deciding whether to trust it.

These algorithms come with attractive worst-case guarantees, yet — as this thesis demonstrates experimentally — their average-case behavior is strikingly muted: on realistic workloads the sophisticated predictions rarely *help*, while the algorithms' real value is that they do not *hurt*. This gap between the worst-case promise and the average-case reality is the puzzle that motivates this thesis. The algorithms had, moreover, only ever been studied *in isolation* — each on its own input model, against its own notion of prediction error, and largely through theory — so there was no common ground on which to compare them, quantify how much a prediction actually buys, or explain the gap. This thesis provides that common ground and, ultimately, an explanation.

1.2 Research objectives

The thesis is organized around three questions:

1. **How do the learning-augmented online-matching algorithms actually compare**, head to head, under a single prediction-error model and on common inputs — and how much does a prediction help, at what cost, on realistic data?

2. **What are the failure modes of the adaptive test-and-fallback mechanism** — the cost of its test, the calibration of its accept/reject decision, and where it goes wrong?
3. **Why does the average-case experience differ so sharply from the worst-case promise** — is the observed wall an artifact of particular algorithms and generators, or is it *necessary*?

The first two are answered experimentally; the third, theoretically.

1.3 Contributions

- **A unified experimental benchmark** (Chapter 4) that places the advice-free baselines, MinPredictedDegree and its augmentations, and the test-and-fallback algorithms on one harness — common graphs, a common structured prediction-error model, a common optimum, and confidence intervals — the first head-to-head comparison of these families.

- **A characterization of predictions as robustness insurance** (Chapters 4, 7): unguarded prediction-following crashes *below* the advice-free baseline under bad predictions; two distinct mechanisms — structural (the augmentations) and adaptive (test-and-fallback) — restore safety with different consistency/robustness trade-offs; and the consistency upside is small on average-case inputs, so the value is downside protection. The picture is validated on synthetic graphs, six real-world graphs, and real request traces, including with a cheap, non-ML historical predictor.

- **An empirical engagement with the order-error theory** (Chapter 5): MinPredicted-Degree's loss is governed by a Kendall-τ order error — the fraction of resource pairs the predictor ranks in the wrong relative order, so that only the ranking a predictor induces matters and not the numbers it outputs — onto which several error models collapse. We credit Aamand–Chen–Indyk's Appendix D for the order-dependence itself and contribute a characterization of the known bound's looseness and saturation and of the right order measure — not a rediscovery that order matters.

- **The first empirical study of test-and-fallback** (Chapter 6): its testing cost, a threshold-calibration pathology in which a *more accurate* test yields a *worse* decision, its recalibration and the resolution limit that recalibration exposes, and a benchmark of the dynamic combiner revealing an irrevocability penalty that explains why matching requires *test-then-commit*.

- **A quantified account of why the wall stands** (Chapter 6). By the *wall* we mean the recurring finding that on average-case inputs neither a better algorithm nor a better prediction moves the competitive ratio appreciably. We trace it to a *resolution limit*: no practical acceptance threshold can capture an upside that sits below the noise of the estimator used to decide. The limit is not one badly calibrated threshold — it persists across the whole difficulty range (Figure 6.4).

Alongside these, the thesis documents (Chapter 8) the exploratory directions it pursued and the negative results they returned — an attempt to *learn* the predictor for extra performance, and an attempt to find a serving regime where predictions genuinely help — both of which reinforce the central finding; the question they raise — is the wall forced? — is taken up in the concluding outlook (§10.2). That outlook is a discussion rather than a further contribution: it proves one consistency/robustness trade-off inequality and then reads our own experiments through it. No theorem beyond that inequality is claimed anywhere in this thesis.

1.4 Thesis organization

Chapter 2 surveys the background: online matching and its input models, the learning-augmented paradigm, the specific prediction-based matching algorithms, and the distribution-testing results behind their tests. Chapter 3 fixes the model and methodology and validates the experimental harness by reproducing a subset of the Borodin et al. study. Chapter 4 presents the unified benchmark and its four findings. Chapter 5 examines what governs the (small) loss — order error — and its relation to the known bound. Chapter 6 studies the test-and-fallback mechanism in depth. Chapter 7 tests external validity on real predictors and real graphs. Chapter 8 reports the exploratory directions and negative results. Chapter 9 presents the AI-inference serving case study. Chapter 10 concludes: it summarizes the findings, gives a theoretical outlook on why the wall is forced (§10.2), and discusses limitations and future work. The recurring thesis, established experimentally and then explained in outlook, is a single sentence: **on average-case online matching, predictions are robustness insurance rather than a performance lever — and the upside they offer is smaller than the price of finding out whether to trust them.**

Chapter 2

Background and Related Work

This chapter sets up the technical context the thesis builds on: the online bipartite-matching problem and its input models (§2.1), the known-i.i.d. model we work in (§2.2), the learning-augmented ("with-predictions") paradigm and its consistency/robustness language (§2.3), the specific prediction-based matching algorithms we benchmark (§2.4), the distribution-testing results behind the algorithms' tests and the outlook of §10.2 (§2.5), and the gaps in the literature this thesis addresses (§2.6).

2.1 Online bipartite matching and competitive analysis

In online bipartite matching, one side of a bipartite graph — the *offline* resources — is known in advance, while the vertices of the other side — the *online* requests — arrive one at a time. On each arrival the algorithm sees the request's incident edges and must either match it to a currently unmatched neighbor or leave it unmatched, **immediately and irrevocably**. The goal is to maximize the size (or, in weighted variants, the value) of the matching. Performance is measured by the **competitive ratio**, the (expected) ratio between the algorithm's matching and an offline optimum on the same input.

The difficulty of the problem depends entirely on the assumed *input model*:

- **Adversarial arrival order.** The requests and their order are chosen by an adversary. The seminal result of Karp, Vazirani and Vazirani [1] is that the randomized **Ranking** algorithm — fix a uniformly random priority order on the resources and match each request to its highest-priority available neighbor — achieves competitive ratio $1 - 1/e \approx 0.632$, and that this is optimal for the adversarial model. Deterministic algorithms cannot beat $1/2$.
- **Random arrival order.** The graph is adversarial but the requests arrive in a uniformly random order; this is strictly easier than adversarial order and admits ratios above $1 - 1/e$.
- **Known-i.i.d.** Requests are drawn independently from a *known* distribution over request types (§2.2); this is the average-case model and the one this thesis studies.

These three models are nested in difficulty, and the direction matters throughout the thesis: every known-i.i.d. instance is also a random-order instance, and every random-order instance is an adversarial-order instance. A guarantee proved in a harder model therefore holds in an easier one — but not conversely, so a bound proved for adversarial order says nothing about how *well* an algorithm does on known-i.i.d. inputs beyond that floor.

2.2 The known-i.i.d. model

In the known-i.i.d. model there is a bipartite *type graph*: each online **type** ℓ has a fixed neighborhood $N(\ell)$ among the offline resources, and an instance is generated by drawing m arrivals independently from a known distribution p over types. The type graph and p are known; the realized sequence is not. This models settings — ad serving, recurring request streams — where the population of requests is statistically stable even though individual arrivals are not.

A line of work has pushed the worst-case (over type graphs) competitive ratio above the $1 - 1/e$ adversarial barrier: Feldman et al. [2] first beat it (0.67) via a flow-based *blue/red* decomposition of a suggested matching; Manshadi, Oveis Gharan and Saberi [3] and Jaillet–Lu [4] improved the ratio (to ≈ 0.702 and ≈ 0.729 respectively) using LP-based and list-based online policies; subsequent work refined it further. These are the "sophisticated" algorithms whose worst-case optimality motivates their use.

Crucially for this thesis, Borodin, Karavasilis and Pankratov [5] conducted an experimental study of these algorithms and found that on *average-case* i.i.d. instances the picture is very different from the worst case: the simple algorithms (Greedy, Ranking) perform almost as well as the sophisticated ones, whose advantage is a worst-case, not a typical-case, phenomenon. This empirical observation — that on typical inputs the simple baseline is already near-optimal — is the seed of the thesis's central finding, and we reproduce a subset of it as validation (Chapter 3).

2.3 Learning-augmented algorithms

The **algorithms-with-predictions** (or *learning-augmented*) paradigm, surveyed in [6], equips an online algorithm with a prediction about the unknown input and asks for two guarantees simultaneously:

- **Consistency:** near-optimal performance when the prediction is accurate;
- **Robustness:** performance no worse than a prediction-free guarantee when the prediction is arbitrarily wrong.

The paradigm was crystallized by Lykouris and Vassilvitskii [7] for competitive caching, who showed how to interpolate between trusting and ignoring a predictor while bounding both consistency and robustness. A large literature followed across ski rental, scheduling, and other online problems, including the *optimal* consistency/robustness trade-off analyses of Wei and Zhang [8], which establish problem-intrinsic Pareto frontiers between the two objectives. A recurring theme is that robustness is obtained by *hedging* — combining the predictor's advice with a safe default so that a bad prediction cannot cause catastrophe. The blind-follow-with-switching **combiner** of Chłędowski, Polak, Szabucki and Żołna [9], introduced in an experimental study of robust learning-augmented caching, is one such hedging mechanism; we port and benchmark it (Chapter 6). That paper is also the closest methodological template for this thesis's experimental half. Recent theoretical work further analyzes how the achievable ratio degrades *continuously* with prediction accuracy in a distributionally-robust formulation [10], complementing the discrete consistency/robustness endpoints used here.

2.4 Predictions for online bipartite matching

Two strands apply the with-predictions paradigm to online matching, differing in the *prediction object* they consume.

Degree predictions: MinPredictedDegree. Aamand, Chen and Indyk [11] propose **MinPredictedDegree (MPD)**: given a prediction μ of each offline resource's degree (how contended it will be), match each arrival to its available neighbor of *minimum predicted degree* — i.e. protect the resources predicted to be rarest. MPD is robust by construction: a constant (useless) predictor reduces it to Ranking. A key structural fact, which the thesis engages in Chapter 5, is that MPD depends on μ *only through the order it induces.* The authors' Appendix D bounds the matching loss by an order quantity built in two steps: list the true degrees in the order the prediction suggests, then count how many of them are out of place — formally $n - \text{LIS}$, where LIS is the length of the longest non-decreasing subsequence of that list. The count is zero exactly when the prediction gets the order right, so a monotone (order-preserving) prediction incurs zero loss.

Type-histogram advice and test-and-fallback. A second strand takes the prediction to be a *histogram* $\hat{c}$ over request types (how many of each type will arrive). Choo et al. [12] introduce **TestAndMatch**: build a matching from $\hat{c}$ and Mimic it, but first spend a sublinear prefix of arrivals *testing* whether the observed type frequencies match $\hat{c}$ — using an ℓ_1-distance tester adapted from Jiao, Han and Weissman [13] — and fall back to Ranking if they do not. Burathep, Erlebach and Moses [14] generalize this ("Test-and-Match+") to the random-arrival model and to imperfect knowledge of the matching size. Both papers give *upper* bounds (algorithms); their only lower bound (Choo et al.'s Theorem 3.1) is a generic *adversarial* indistinguishability result (no algorithm is 1-consistent and $> \frac{1}{2}$-robust), and neither proves a lower bound in the stochastic model. Notably, Choo et al.'s acceptance threshold already *couples* to the baseline competitive ratio β — but constructively, inside the algorithm design, not as a lower bound. What that coupling costs — how large a prefix the follow/fallback decision fundamentally requires — is a question this thesis engages empirically (Chapter 6) and in a theoretical outlook (§10.2); the full formal development is deferred to companion work.

2.5 Distribution testing

The test at the heart of the algorithms above is a question in **distribution testing**: given samples from an unknown distribution p over a support of size r and a known reference q, decide how far p is from q. We measure that distance in ℓ_1 throughout, as the algorithms and our experiments do; it is *twice* the total-variation distance, and every threshold quoted in this thesis is an ℓ_1 threshold. Two regimes must be distinguished:

- **Identity (non-tolerant) testing** — distinguish $p = q$ from $\|p - q\|_1 \geq \varepsilon$ — has sample complexity $\Theta(\sqrt{r}/\varepsilon^2)$ [15, 16], *sublinear* in the support size.
- **Tolerant testing / distance estimation** — distinguish $\|p - q\|_1 \leq \varepsilon_1$ from $\geq \varepsilon_2$ for $0 < \varepsilon_1 < \varepsilon_2$, i.e. estimate the distance rather than test equality — is provably *much harder*: Valiant and Valiant [17] showed it requires $\Theta(r/\log r)$ samples, *near-linear* in the support. Jiao, Han and Weissman [13] gave matching bounds for ℓ_1-distance estimation, and Canonne, Jain, Kamath and Li [18] precisely characterized the whole $(\varepsilon_1, \varepsilon_2)$ landscape, showing that for constant tolerances the cost jumps to the "barely sublinear" $\widetilde{\Theta}(r/\log r)$.

The near-quadratic gap between $\sqrt{r}$ (testing) and $r/\log r$ (tolerant testing) matters twice in this thesis. It is why the deployable versions of TestAndMatch fall back to an empirical surrogate for their tester, and it is why that surrogate is blind on large supports — the resolution limit measured in Chapter 6. Whether the barrier also dooms every *other* decision rule turns out to be subtler than it first appears; the concluding outlook (§10.2) returns to this point.

2.6 Positioning of this thesis

Against this background, three gaps stand out, which the thesis addresses in turn:

1. **No unified empirical comparison.** The matching algorithms above were each studied in isolation, on their own input families and error models, and largely in theory; there is no head-to-head experimental benchmark under a common harness. (Chapters 4–7.)
2. **No empirical study of test-and-fallback.** The distribution test at the heart of [12, 14] has no deployable implementation (the authors themselves fall back to an empirical surrogate), and its testing cost, threshold calibration, and failure modes have not been measured. (Chapter 6.)
3. **No quantification of what the prefix test costs.** No prior work measures — or bounds — how large a prefix the follow/fallback decision requires on strong-baseline instances, where the upside to be captured is smallest. (Chapter 6 empirically; §10.2 in outlook.)

The thesis closes the first two gaps experimentally and takes a first, quantified step at the third — an empirical resolution limit plus a theoretical outlook — arriving at a single unifying statement: *on average-case online matching, predictions are robustness insurance rather than a performance lever*, supported by experiments across synthetic graphs, real graphs, and real traces.

Chapter 3

Model, Algorithms, and Methodology

Chapter 2 placed the problem in its field. This chapter fixes the precise model and notation used throughout (§3.1), details the algorithms as we implement them (§3.2), the prediction objects and structured error models (§3.3), the consistency/robustness measures (§3.4), and the experimental harness (§3.5). It closes by validating the harness against the published Borodin et al. figures before any contribution is built on it (§3.6).

3.1 The known-i.i.d. model, precisely

There is a set R of n offline **resources** and a **type graph** on r online **types**; type ℓ has neighborhood $N(\ell) \subseteq R$. An instance is generated by drawing m arrivals i.i.d. from a distribution p over the r types; the i-th arrival reveals its type (hence $N(\cdot)$) and must be matched to a currently unmatched resource in its neighborhood, immediately and irrevocably, or left unmatched. The type graph and p are known to the algorithm; the realized arrival sequence is not. Following the standard *integral-types* convention, the default is $m = n$ with uniform p, so each type's expected count is an integer; the classical setting has $r = n$ (each offline vertex a distinct type), and the *few-types* setting used for the histogram-advice experiments has $r \ll n$.

The performance measure is the **competitive ratio** $\rho(\text{ALG}) = \mathbb{E}[|\text{ALG}|]/\mathbb{E}[|\text{OPT}|]$, where OPT is a maximum matching of the *realized* instance, computed exactly by Hopcroft–Karp [19]. We report the ratio of expectations rather than the expectation of the ratio, following Borodin et al. [5]; the two differ in general, and the former is what allows a single exactly-computed OPT per instance to be shared by every algorithm in a paired trial (§3.5). The advice-free baseline throughout is KVV Ranking, whose ratio we write ρ_{base} (the **baseline strength**).

The notation recurs throughout the thesis and is collected here for reference:

symbol	meaning	typical value
n	offline resources	1000–2000
r	online request types	$r = n$ (classical), $r = 8$ (few-types)
m	arrivals in one instance	$m = n$
p	known distribution over types	uniform unless stated

symbol	meaning	typical value
μ	predicted resource degrees (degree prediction)	—
$\hat{c}$	predicted type counts (histogram prediction)	—
k	length of the prefix a test may inspect	$k = 200$
η	strength of the injected prediction error	$\eta \in [0, 1]$
ρ_{base}	Ranking's ratio on that instance family (the floor)	0.89–0.99

3.2 Algorithms

All greedy-type algorithms are instances of one primitive: given a total order (**rank**) on the resources, **GreedyWithPermutation** matches each arrival to its available neighbor of minimum rank. This makes the algorithm family a single code path parameterized by the rank, which is important both for a fair comparison and for the theory.

- **Advice-free baselines.** SimpleGreedy (identity rank); **Ranking** (uniformly random rank; the baseline); and the stochastic-matching algorithms Feldman and Jaillet–Lu, which precompute a suggested matching by max-flow — Feldman via a cap-$\{2, 1, 2\}$ flow network and a blue/red decomposition of the unit-flow subgraph, Jaillet–Lu via a cap-$\{3, 2, 3\}$ network yielding a fractional solution $f^\star \in \{0, \frac{1}{3}, \frac{2}{3}\}$ and per-type list distributions. Each has a *non-greedy* variant (follow the suggestion only) and a *greedy* variant (fall back to any available neighbor).
- **Degree predictions.** MinPredictedDegree (MPD) is GreedyWithPermutation ranked by ascending predicted degree $\mu \in \mathbb{R}^R$. Constant μ gives Ranking; the true realized degrees give the consistency ceiling (MinDegree). The **augmentations** Feldman(MPD) and JailletLu(MPD) use the MPD rank only as the tie-break of the greedy fallback, so the worst-case-optimal base matching carries the load.
- **Type-histogram advice and test-and-fallback.** From a count vector $\hat{c}$ over types we build an advice matching $\widehat{M}$ (a maximum matching of $\hat{c}$ copies) and record its per-type partners. **FollowPrediction** *mimics* $\widehat{M}$ — routes every arrival to its type's advice partner; **TestAndMatch** (Choo and BEM variants) *mimics* over a length-k prefix, tests the empirical ℓ_1 distance between the prefix type frequencies and $q = \hat{c}/\|\hat{c}\|_1$ against a threshold τ, then continues or falls back to Ranking. We also port the Chłędowski-style dynamic **combiner** as a benchmarked baseline (not a contribution).

3.3 Prediction objects and structured error models

The families consume different prediction objects — degree vectors μ and type histograms $\hat{c}$ — so each is corrupted by its own knob and reported in a parallel panel. For μ we use four structured error models, each driven by a strength $\eta \in [0, 1]$ and each returning both an ℓ_1 *magnitude* error and a normalized **Kendall-τ order error**:

model	construction at strength η	effect on the induced order
random-flip	independently, with probability η, replace an entry of μ by a uniform draw from $[\min\mu, \max\mu]$	grows with η; $\eta = 1$ is a constant-quality predictor, i.e. Ranking
systematic-bias	monotone rescale $\mu \leftarrow \mu \cdot (1+\eta)$	none — Kendall-$\tau \equiv 0$ by construction, while the ℓ_1 magnitude grows linearly
adversarial	reflect an η-fraction of entries, $\mu(R) \leftarrow (\min\mu + \max\mu) - \mu(R)$	the most order-damaging perturbation at a given fraction; $\eta = 1$ reverses the order
distribution-drift	blend toward the degrees of an independently drawn graph of the same family, $\mu \leftarrow (1-\eta)\mu + \eta\mu_{\text{alt}}$	grows with η, but in a correlated way rather than entry-by-entry

For $\hat{c}$ we blend the realized counts toward a concentrated random target by a fraction $\eta \in [0, 1]$, reporting the induced $\ell_1(p, q)$. Because MPD depends on μ only through the induced order, the order/magnitude distinction is the axis of Chapter 5.

3.4 Consistency and robustness

For a prediction-consuming algorithm, **consistency** is its ratio under perfect advice and **robustness** is its worst-case ratio under adversarial advice. A useful algorithm is consistent *and* never (much) below ρ_{base}; the tension between the two is the subject of Chapter 6 and of the outlook in §10.2.

3.5 The experimental harness

The harness is a small, dependency-light Python codebase (NumPy, SciPy, NetworkX). All randomness comes from independent, reproducible streams obtained by NumPy's spawn mechanism — by default four streams (graph, instance, Ranking, Jaillet–Lu), with additional streams spawned for prediction generation and perturbation, so adding an algorithm never disturbs existing results. We use **paired trials**: within a comparison, every algorithm and error level reuses the same graph, arrival sequence, OPT, and tie-break seed, so differences are attributable to the prediction alone. Reported ratios are means over trials with 95% normal-approximation confidence intervals, computed per algorithm and error level. Because the trials are paired, those per-algorithm intervals are *conservative* for the comparisons we actually draw: the algorithms' ratios are positively correlated by construction, so the standard error of a difference between two of them is smaller than that of either mean, and two non-overlapping intervals remain a valid — if pessimistic — verdict on the paired difference. Where a comparison is close enough for this to matter, we say so in the text rather than lean on the intervals. Every figure and table in the thesis is regenerated from a fixed seed by a single script (Appendix A).

3.6 Fidelity check: reproducing Borodin et al.

Before building prediction-based algorithms on the harness, we validated it against the published experimental study of Borodin, Karavasilis and Pankratov [5]. This is a **fidelity check, not a contribution**: its purpose is to confirm that the generators, the i.i.d. sampler, the Hopcroft–Karp optimum, and the algorithm implementations reproduce the paper's qualitative findings, so that later differences can be attributed to the algorithms rather than to infrastructure. Our stack (Python/NetworkX) differs from the paper's (C++/Edmonds–Karp), so we target *qualitative* agreement, accepting small absolute differences (≤ 0.02).

We reproduced the core four algorithms (six greedy/non-greedy variants) on two random graph families at $n = 1000$, $m = n$, 100 trials per parameter value (matching the paper's setup), under a single master seed.

Erdős–Rényi (edge probability c/n; the paper's Fig. 9, partial). Sweeping c reproduces the characteristic U-shape and its hard case: SimpleGreedy attains its minimum 0.864 at $c \approx 4.9$, and the greedy variants of the complex algorithms their minima ≈ 0.884 at $c \approx 5.3$. Ranking is indistinguishable from SimpleGreedy (maximum difference 0.0017 across all 75 values — the paper omits Ranking's curve for exactly this reason), and the non-greedy variants degrade monotonically as c grows ($0.987 \to 0.729$ for Feldman, $0.985 \to 0.764$ for Jaillet–Lu). Every one of the paper's five prose claims we checked holds.

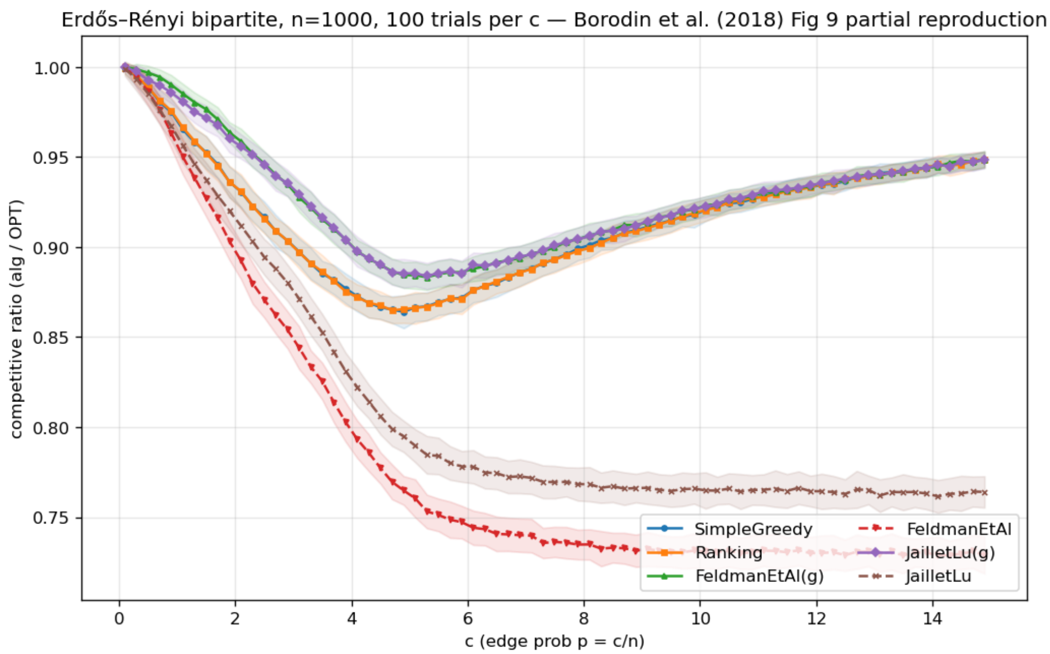


Figure 3.1: Erdős–Rényi reproduction: the characteristic U-shape, greedy minima near $c \approx 4.9$, non-greedy degradation as c grows.

Random left-regular (left-degree d; the paper's Fig. 18, partial). The pattern repeats with the hard case at $d = 5$ (SimpleGreedy minimum 0.890), Ranking $\approx$ SimpleGreedy (max difference 0.005), non-greedy degradation as d grows, and greedy complex variants $\approx$ SimpleGreedy asymptotically.

A cross-family observation. Combining the two sweeps surfaces a non-obvious fact: the non-

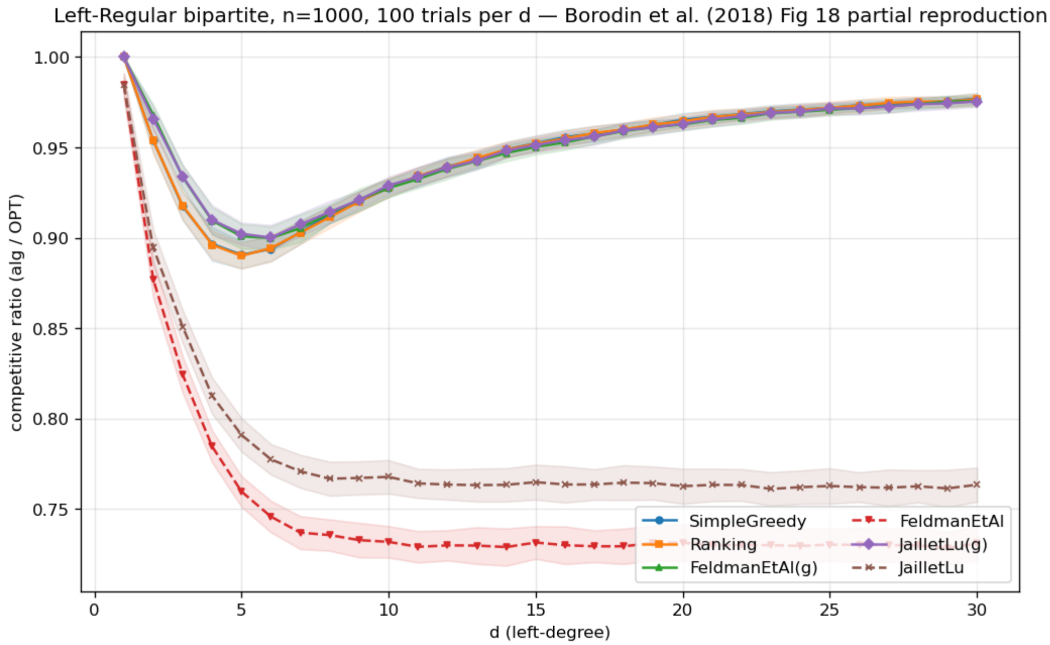


Figure 3.2: Random left-regular reproduction: the hard case at $d = 5$, Greedy $\approx$ Ranking.

greedy Feldman and Jaillet–Lu algorithms converge to the *same* asymptotic ratio in *both* families — 0.729 and 0.764 respectively, within 0.002 across families — and both sit *above* their worst-case theoretical bounds (0.670 and 0.729) by +0.06 and +0.03. This hints at a universal "average-case asymptotic constant" distinct from the worst-case guarantee — an early, concrete instance of the thesis's recurring theme that the average case is far more benign than the worst case, which the prediction-error sweeps of later chapters probe in earnest.

Verdict. The harness reproduces the published qualitative findings on both families, including the locations of the hard cases and the near-identity of Greedy and Ranking. The foundation is trusted; the contributions of Chapters 4–9 are built on it. (Full reproduction tables and the paper-claim checklist are in Appendix A.)

Chapter 4

The Unified Benchmark

Having validated the harness (Chapter 3), we now place all three algorithm families on it and establish the thesis's organizing finding. This chapter reports competitive ratios (mean $\pm$ 95% CI) as a function of prediction quality, in three panels chosen to span the regimes each family was designed for; the four findings it draws out (§4.2) recur through the rest of the thesis.

4.1 Design

The families consume different prediction objects, so each is driven by its own corruption knob and reported in a parallel panel (Chapter 3); the shared harness — graphs, OPT, CI methodology, and the advice-free floor — is what makes the panels comparable.

Instance format and notation. Every panel uses the instance format of Chapter 3: n offline resources, r online request types, and m arrivals drawn i.i.d. from the type distribution; throughout this chapter $m = n$, so requests and resources are balanced. The panels differ *only* in the type graph connecting requests to resources — that single change is what moves the input between the regimes the two prediction families were designed for:

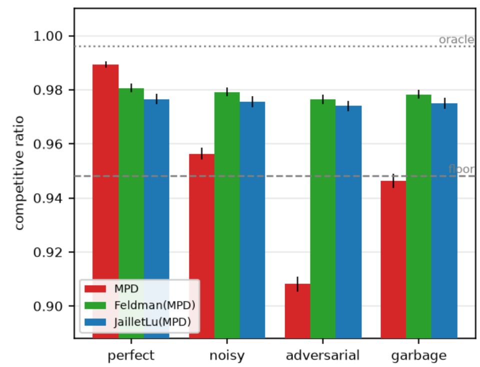
- **Panel A — clvb_zipf** ($n = 1000$, 60 trials): resource degrees follow a Zipf power law with exponent 1.0 — a few resources are heavily contended while most are rarely eligible. This heavy-tailed profile gives a *degree* predictor genuine signal to carry.
- **Panel B — left-regular** d=5 ($n = 1000$, 60 trials): each arriving request connects to exactly $d = 5$ uniformly random resources, so resource degrees are nearly homogeneous — the hard case of Chapter 3, where a degree predictor has almost no signal left to carry.
- **Panel C — few-types** r=8 ($n = 2000$, 50 trials): only $r = 8$ distinct request types, each arriving $\approx n/r = 250$ times on average — the near-perfect-matchable, few-types regime the *histogram*-advice algorithms are calibrated for; their test-and-fallback test inspects a prefix of $k = 200$ arrivals.

Panels A and B thus exercise the degree-prediction family (MPD and its augmentations); Panel C exercises the histogram-advice family (FollowPrediction, TestAndMatch, and the combiner).

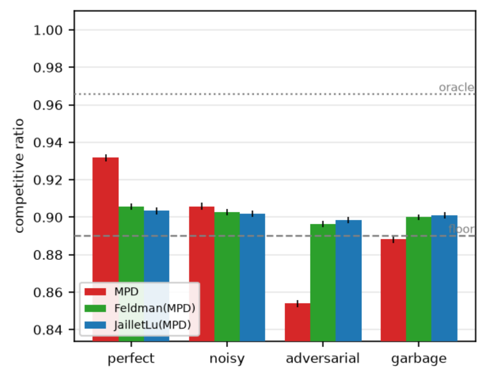
Shared methodology. Paired trials, independent random streams and confidence intervals are exactly as in §3.5; the panel-specific parameters are the ones listed above, and the intervals are tight throughout ($\pm$0.001–0.003). The quality columns instantiate the error models of §3.3 —

degree panels: *perfect* (true realized degrees), *noisy* (random-flip at strength $\frac{1}{2}$), *adversarial* (order-reversing reflection), *garbage* (independent random μ, $\equiv$ Ranking); advice panel: the true histogram blended toward a concentrated random target by $\eta \in \{0, 0.3, 0.6, 1.0\}$ (*perfect* / *mild* / *bad* / *garbage*). The two sets of columns are *not* commensurable — they corrupt different prediction objects with different knobs — so only within-panel comparisons carry meaning. **Table 4.1** presents each panel's ratios beside its bar chart; the findings follow in §4.2.

Panel A — clvb_zipf	perfect	noisy	advers.	garbage
Ranking (floor)	0.948	—	—	—
MinDegree (oracle)	0.996	—	—	—
MPD	0.989	0.956	**0.908**	0.946
Feldman(MPD)	0.981	0.979	0.976	0.978
JailletLu(MPD)	0.977	0.976	0.974	0.975
Feldman (base)	0.887	—	—	—
JailletLu (base)	0.901	—	—	—



Panel B — left-regular d=5	perfect	noisy	advers.	garbage
Greedy = Ranking (floor)	0.890	—	—	—
MinDegree (oracle)	0.966	—	—	—
MPD	0.932	0.906	**0.854**	0.888
Feldman(MPD)	0.906	0.903	0.896	0.900
JailletLu(MPD)	0.903	0.902	0.898	0.901
Feldman (base)	0.758	—	—	—
JailletLu (base)	0.789	—	—	—



Panel C — few-types r=8	perfect	mild	bad	garbage
Ranking (floor)	0.990	—	—	—
MinDegree (oracle)	0.999	—	—	—
FollowPrediction	1.000	0.832	0.679	**0.472**
TestAndMatch (Choo)	1.000	0.984	0.989	0.990
TestAndMatch (BEM)	0.998	0.988	0.988	0.968
Combiner *(benchmark)*	0.990	0.990	0.990	0.990

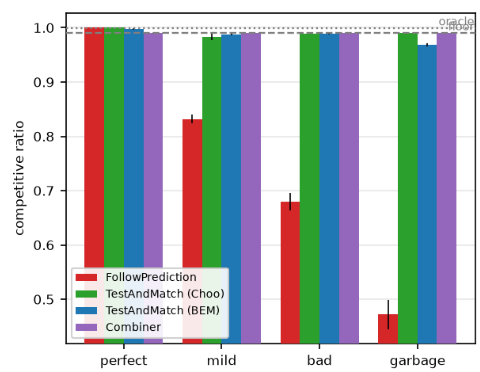


Table 4.1: The unified benchmark. Each panel's competitive ratios (means over paired trials; every 95% CI ≤ 0.003) sit beside their bar chart (error bars: 95% CIs; dashed line: advice-free floor; dotted: oracle ceiling). Bold marks the worst column of each unguarded prediction-follower; all three fall below their own panel's floor. That floor is instance-dependent — it is Ranking's ratio on that panel's graph family, not a constant — so the three floors (0.948, 0.890, 0.990) are not comparable with one another, and neither are the quality columns across the two prediction families (§3.3).

4.2 Four findings

(F1) Robustness is engineered, not free: naive followers crash below the floor. Both unguarded prediction-followers dive under the advice-free Ranking floor once the prediction is adversarial or garbage: MPD falls to $0.908 < 0.948$ (Panel A) and $0.854 < 0.890$ (Panel B), and FollowPrediction collapses to $0.472 \ll 0.990$ (Panel C). Under adversarial or garbage advice, then, using either *unguarded* is worse than using no prediction at all. Every robust algorithm — the augmentations, TestAndMatch, the combiner — avoids this by construction.

(F2) Two distinct robustness mechanisms, with different shapes. *Structural* robustness (Feldman(MPD), JailletLu(MPD)): the worst-case-optimal base matching carries the load and the prediction only breaks ties, so performance is nearly *flat* — Feldman(MPD) moves only $0.981 \to 0.976$ from perfect to adversarial (Panel A). It cannot crash but caps the upside (never reaching the 0.996 oracle). *Adaptive* robustness (TestAndMatch): test a sublinear prefix, then commit — capturing the upside when advice is good (Choo 1.000) and holding the floor when it is bad (0.990). On Panel C it is the only algorithm on the upper envelope at both ends. The two mechanisms trade consistency for robustness in opposite ways; **Figure 4.1** plots every algorithm on the consistency–robustness plane, where the opposite trades — and the empty region beyond TestAndMatch toward the ideal top-right corner — are visible at a glance.

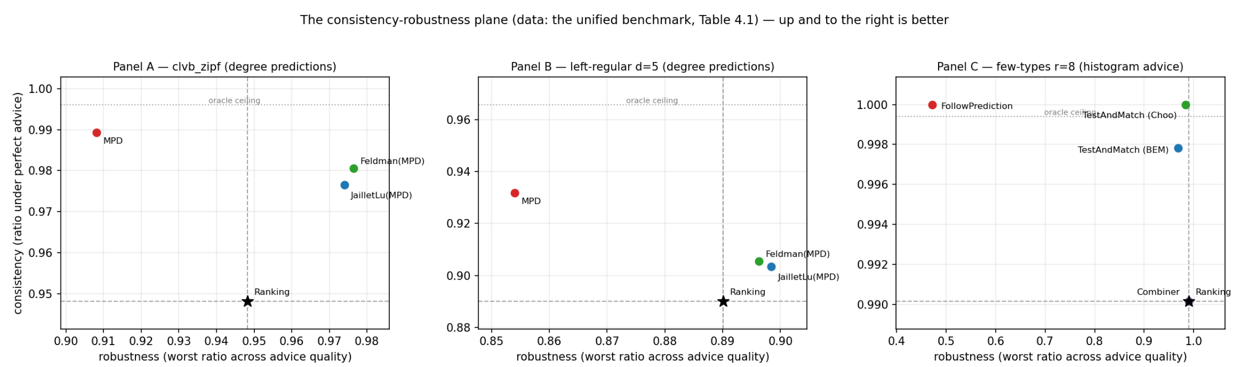


Figure 4.1: The consistency–robustness plane (the data of Table 4.1); dashed lines mark the advice-free floor, dotted the oracle ceiling. TestAndMatch sits nearest the ideal top-right corner.

(F3) The consistency upside is small on average-case inputs; the spread lives on the bad-advice side. On few-types the advice-free Ranking is already 0.990 and MPD-with-true-degrees is 0.999 — under 0.01 for any advice to add on the good side. Every wide gap in Panel C is a *downside* gap. This is the thesis in one panel; Chapter 6 shows why no affordable test escapes it, and the concluding outlook (§10.2) why it is forced.

(F4) The augmentation rescues structurally weak base algorithms. Feldman and Jaillet–Lu are tuned for the worst-case ratio and are the *weakest* advice-free entries on these average-case inputs (Panel B: 0.758 / 0.789, below Greedy's 0.890); the MPD augmentation lifts them to ≈ 0.90. The prediction does *more* for the worst-case-designed algorithms than for greedy — a pairing visible only under a unified table.

4.3 Chapter summary

On average-case matching the advice-free baseline is already near-optimal, unguarded prediction-following is unsafe, and the value of the sophisticated algorithms is downside protection delivered by one of two mechanisms. Chapters 5–7 sharpen each part — what governs the (small) loss, what the adaptive test costs, and whether the picture survives on real data — and the concluding outlook (§10.2) explains why the wall is forced, not accidental.

Chapter 5

What Governs the Loss: Order Error and ACI's Bound

Chapter 4 showed that on average-case inputs the loss of a degree-prediction algorithm is small. This chapter asks what *governs* that loss. MinPredictedDegree matches by ascending predicted degree, so it depends on the predictor μ *only through the order it induces* on the resources: two predictors inducing the same order produce identical matchings, and a monotone rescaling of μ changes nothing. The right question is therefore not "how large is the error" but "which *order* error governs the loss, and how tightly." Answering it requires care, because the qualitative fact that order — not magnitude — matters is already a theorem, and we are explicit about what is prior and what is ours.

5.1 What is already known (ACI)

Aamand, Chen and Indyk [11, Appendix D] prove that on the CLV-B model, MinPredictedDegree's matching loss relative to the true expected degrees is at most $n - \mathrm{LIS}(w[\mu])$. Here w is the vector of true expected degrees — written w rather than p to keep it distinct from the type distribution p of §3.1 — $w[\mu]$ is that vector listed in the order μ induces, and LIS is the length of its longest non-decreasing subsequence: a pure *order* quantity. In particular a monotone (order-preserving) predictor leaves $w[\mu]$ already sorted, so $n - \mathrm{LIS} = 0$ and the loss is zero. Thus *order-dependence* and *the zero-effect of a monotone bias* are ACI's results, not ours; our `systematic_bias` error model (a monotone rescale) has Kendall-$\tau \equiv 0$ by construction and, consistently, leaves MPD's ratio exactly unchanged across the benchmark (Chapter 4) — an empirical confirmation of ACI's statement, not a new finding.

5.2 Our characterization

We sweep the four structured error models across strength and record, per model and level on the same instances, three quantities: the actual MPD matching loss, ACI's $n - \mathrm{LIS}$, and the normalized Kendall-τ order error (**Figure 5.1**; $n = 1000$, Zipf exponent 1.0, 40 trials).

(i) ACI's $n - \mathrm{LIS}$ bound is correct but very loose. The realized loss lies far below the bound at every point — by roughly 16× (adversarial) to 75× (distribution-drift); all points hug the axis in Figure 5.1(a).

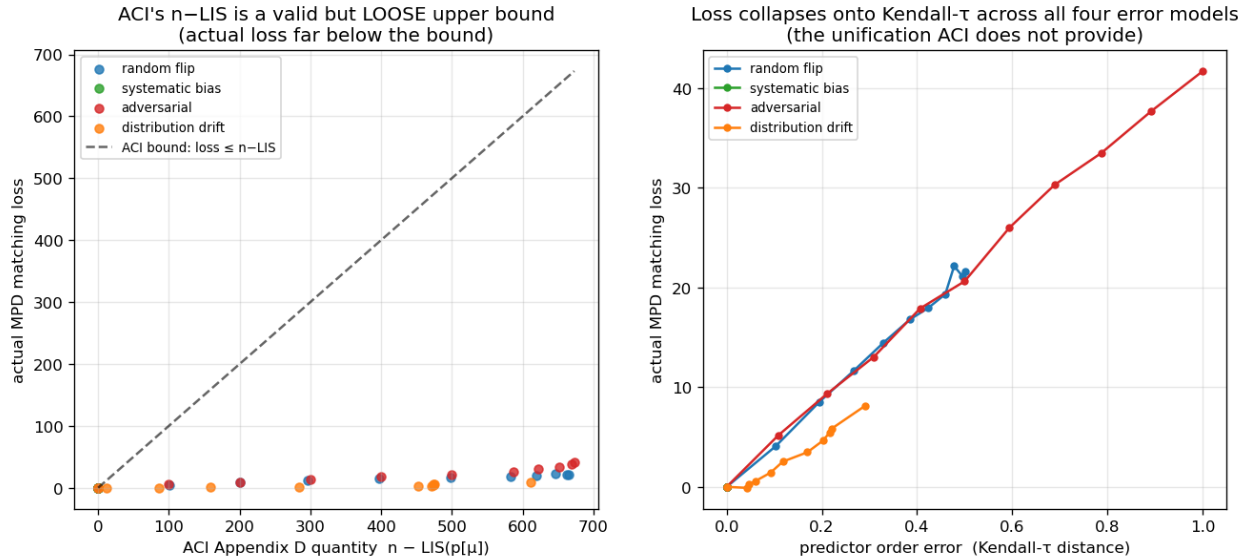


Figure 5.1: Order error governs the loss: the realized loss lies far below ACI's $n - \mathrm{LIS}$ bound (a) and collapses onto Kendall-τ across all four error models (b).

(ii) $n - \mathrm{LIS}$ saturates and cannot distinguish error structures. For every non-trivial error it is pinned near its maximum ($\approx$ 610–673 for $n = 1000$), even though the true losses differ by a factor of five across models (8.1 drift, 21.6 random-flip, 41.7 adversarial). As a bound that is almost always $\approx n$, it carries little information about which prediction is more harmful.

(iii) Kendall-τ is the governing order measure, and the models collapse onto it. Plotted against Kendall-τ (Figure 5.1(b)), the four models fall on one increasing curve — loss rises with τ ($0.29 \to 0.50 \to 1.0$ for drift, random-flip, adversarial, tracking $8.1 \to 21.6 \to 41.7$), with `systematic_bias` pinned at $\tau = 0$, zero loss. The collapse is not only visual: pooling all four models and all error levels (44 points), the rank correlation between Kendall-τ and the realized loss is Spearman $\rho = 0.979$ (Pearson $r = 0.992$), and within each non-degenerate model it is 0.97 or above. The quantity that *predicts* the loss and unifies the error structures is the Kendall-τ order distance, which the saturated $n - \mathrm{LIS}$ cannot resolve.

5.3 Chapter summary

We do not claim that order rather than magnitude governs MPD — ACI's Appendix D establishes that, along with the zero-effect of a monotone bias. What we add is a precise empirical characterization: ACI's $n - \mathrm{LIS}$ bound is loose by one to nearly two orders of magnitude and saturates, whereas Kendall-τ predicts the realized loss and unifies the four error structures. This both sharpens the theory's practical content and justifies the single order-error axis used when the predictor is stressed on real data (Chapter 7).

Chapter 6

Test-and-Fallback in Depth

The test-and-fallback algorithms are the adaptive robustness mechanism of Chapter 4. This chapter gives their first empirical study: the robustness envelope they achieve (§6.1), a counter-intuitive failure of the acceptance threshold (§6.2), its recalibration and the resolution limit that recalibration exposes (§6.3) — a limit that persists across the whole difficulty range and anchors the theoretical outlook of §10.2 — and a benchmark of the dynamic combiner that explains the *commit-once* structure (§6.4). Two naming conventions apply throughout. **FollowPrediction** is the unguarded follower, which mimics the advice matching without testing it; **TestAndMatch** is the test-and-fallback scheme in general, of which **Choo** and **BEM** are the two published instantiations — they share the test-then-commit structure and differ in how the acceptance threshold is set. The ℓ_1 test is the empirical surrogate the original authors also fall back to (Chapter 3).

6.1 The robustness envelope

Sweeping advice error $\ell_1(p, q)$ on few-types instances ($n = 2000$, $r = 8$, prefix $k = 200$, 40 trials; **Figure 6.1**), FollowPrediction degrades *linearly* from 1.000 at perfect advice to 0.453 at $\ell_1 \approx 1.1$ — well *below* the advice-free floor (≈ 0.99). TestAndMatch instead stays on the **upper envelope**: it captures the benefit when advice is good and, when advice is bad, its prefix test rejects it and it falls back to Ranking, never crashing (BEM 0.998 $\to$ 0.969; Choo 1.000 $\to$ 0.991). This is the adaptive counterpart of the structural robustness of Chapter 4.

6.2 A threshold that is too lenient on average-case inputs

Sweeping the prefix (testing) size k at *borderline* advice ($\eta = 0.15$, true $\ell_1 \approx 0.16$) — where the test works hardest (**Figure 6.2**) — a larger, more accurate test makes the *worse* decision: as k grows $25 \to 800$, the ratio falls $0.992 \to 0.956$ and the misjudgement rate rises $0.00 \to 0.60$.

The mechanism has two halves. First, the acceptance threshold τ is calibrated to β, the competitive ratio the advice-free baseline is *proved* to achieve — here $\beta \approx 0.696$, Ranking's worst-case ratio under random arrival order, the value Choo et al. instantiate their threshold with [12]. On these instances the realized baseline is instead ≈ 0.99 (F3), so the empirical break-even sits at $\ell_1 \approx 0$, far below τ. Second, the prefix interacts with that gap in opposite directions at the two ends of the sweep. A small, noisy prefix over-estimates ℓ_1 and rejects the borderline advice, landing safely on the floor — it is right for the wrong reason, rejecting because it is noisy rather than because the

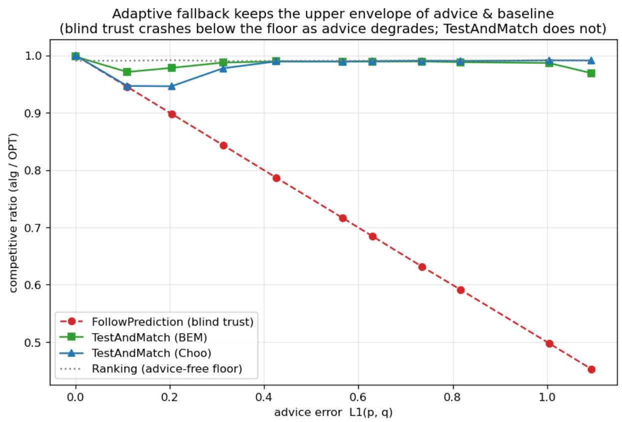


Figure 6.1: The robustness envelope: blind FollowPrediction crashes below the advice-free floor as advice error grows; TestAndMatch stays on the upper envelope.

advice is bad. A large, accurate prefix measures $\ell_1 \approx 0.16 < \tau$ correctly and therefore accepts the mildly-bad advice that the worst-case threshold deems acceptable, underperforming the baseline. On strong-baseline inputs the worst-case threshold is too lenient, and a more accurate test only follows it more faithfully.

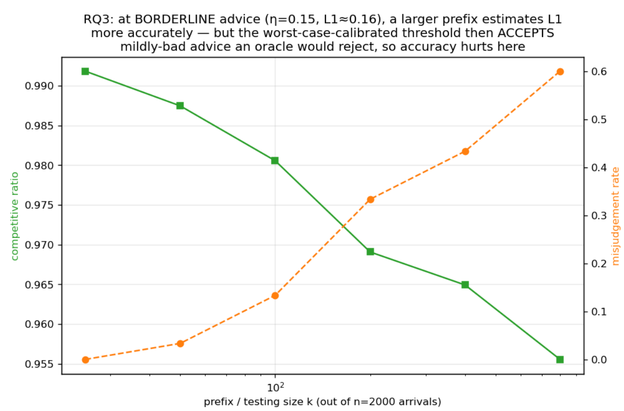


Figure 6.2: Testing cost at borderline advice: a larger, more accurate prefix test makes the *worse* decision under the worst-case-calibrated threshold.

6.3 Recalibration, and the resolution limit it exposes

By the **resolution** of a test we mean the smallest difference in ℓ_1 that its empirical estimator can distinguish from its own sampling noise at prefix length k; this section is about that quantity and the limit it imposes. Recalibrating τ to the *measured* baseline $\widehat{\beta}$ eliminates the pathology (**Figure 6.3**): at borderline advice the worst-case threshold's misjudgement climbs $0.03 \to 1.00$ as the prefix grows and its ratio drops to 0.920, while the recalibrated threshold holds misjudgement 0.00 at every prefix and ratio ≈ 0.986. But recalibration exposes a deeper limit. At perfect advice the worst-case threshold scores 1.000 while the recalibrated one scores only 0.987: the recalibrated $\tau \approx 2(1 - \widehat{\beta}) \approx 0.028$ is *smaller than the empirical-ℓ_1 estimator's noise floor* ($\approx$ 0.05–0.13), so it can never confidently *accept* — it rejects everything, including perfect advice, and always plays the baseline. In short:

> On strong-baseline instances, no practical empirical-ℓ_1 threshold can both capture the consistency upside and stay safe: the upside is tiny and sits *below the estimator's resolution*. The worst-case threshold over-accepts; the recalibrated one over-rejects; a better tester only follows whichever threshold more faithfully.

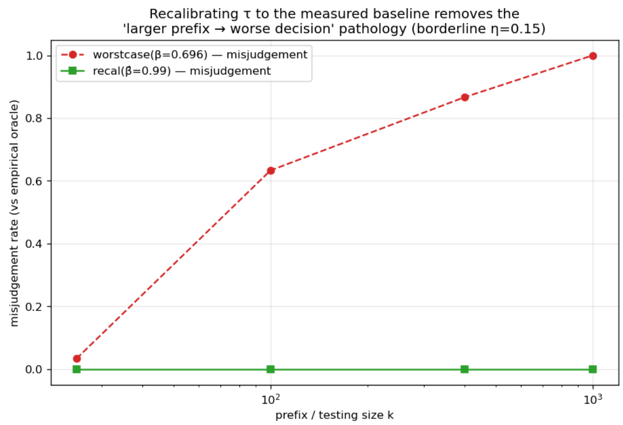


Figure 6.3: Recalibration removes the threshold pathology: misjudgement holds at zero at every prefix size, versus climbing toward 1.0 under the worst-case threshold.

Figure 6.4 widens this finding from one threshold to the whole difficulty range. Sweeping the number of types — the knob that sets the baseline strength — the *potential* upside of perfect advice grows as the baseline weakens, but the upside a sublinear-prefix test can *safely capture* stays pinned near zero: the empirical test's resolution (its noise floor) sits far above the break-even margin wherever an upside exists. The wall of this chapter is therefore not one badly calibrated threshold. The upside and the testing resolution move together — and the concluding outlook (§10.2) gives this coupling a quantitative form: the prefix needed to decide scales as the inverse square of the stakes.

6.4 The dynamic combiner is dominated, and shows why matching needs test-then-commit

We benchmark the Chłędowski-style dynamic combiner to contextualize the commit-once structure of test-and-fallback. In its robust tuning the combiner sits exactly on the floor of the few-types family of §6.1 (0.990 across all advice quality): it never crashes but captures none of the consistency upside, strictly dominated by TestAndMatch. More instructively, an *eager* combiner that switches mid-stream reveals a penalty specific to irrevocable problems. On a smaller instance used as a mechanism check — $n = 600$, $r = 6$, a single seed with no averaging, one of the hand-verifiable scripts of Appendix A.3 — eager switching under perfect advice scores 0.927, below both the pure follower (1.000) and Ranking on that same instance (0.958), because switching from Ranking to advice-following mid-run lands the committed matching in an *incompatible hybrid*. Being one instance, this figure illustrates the mechanism rather than measuring its size; note also that its 0.958 is Ranking on *that* instance, not the 0.990 floor of the family above. In an irrevocable problem the follow/fallback decision must be made *before* the bulk of the commitments, which is why Choo/BEM test a prefix and then *commit* rather than switching dynamically. The dynamic

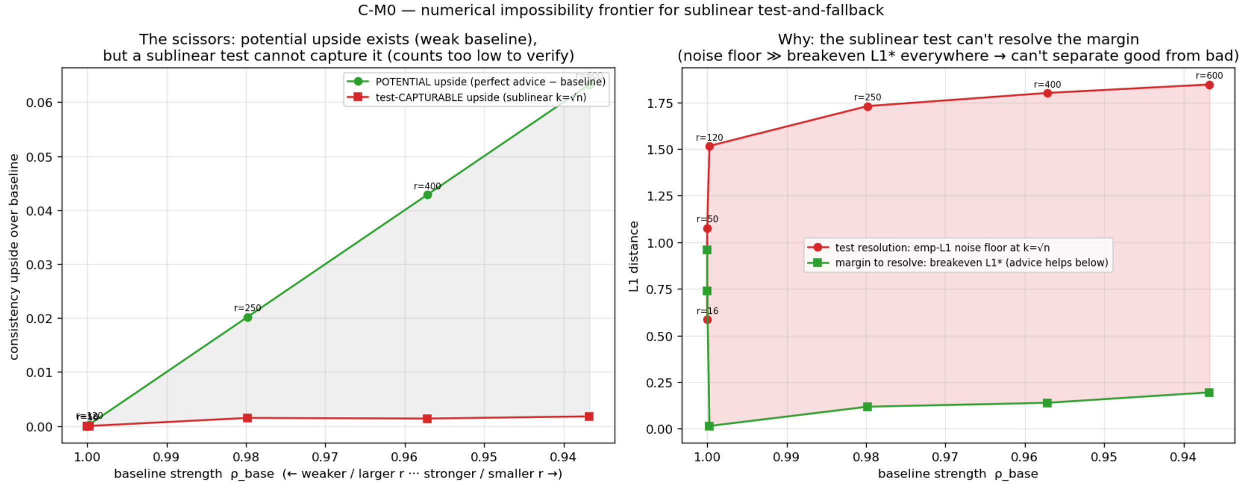


Figure 6.4: The testing-wall frontier: as the baseline weakens (left), the *potential* upside of perfect advice grows, but the upside a sublinear test can *safely capture* stays near zero — the empirical test's resolution sits far above the break-even margin wherever the upside exists.

combiner that is cheap insurance for caching does not port cleanly to matching.

6.5 Chapter summary

Test-and-fallback delivers the adaptive robustness envelope of §6.1, but its accept/reject decision is fundamentally limited on strong-baseline inputs: no empirical-ℓ_1 threshold both captures the upside and stays safe (§6.3), and dynamic switching is dominated by test-then-commit (§6.4). The resolution limit of §6.3 — and its persistence across the whole difficulty range (Figure 6.4) — is the empirical anchor of the theoretical outlook in §10.2.

Chapter 7

External Validity

Chapters 4–6 use synthetic graphs and a synthetic error knob. Is the picture real? We stress it three ways: a genuine, cheap predictor on a real request trace (§7.1); the six real-world graphs (§7.2); and whether *learning* the predictor changes anything (§7.3).

7.1 A real, cheap predictor

We replace the synthetic knob with the cheapest realistic predictor: last-window historical statistics. Real Wikipedia daily pageviews give a live day (the truth) and earlier days (a 1-, 7-, or 30-day-stale forecast), so the error is genuine temporal drift; we map the trace onto a fixed serving topology and consume the forecast through the degree route (MPD) (**Figure 7.1**). Throughout this chapter we measure how much of the available benefit a predictor realizes by its **gap-capture**, $(\rho_{\mathrm{ALG}} - \rho_{\mathrm{base}})/(\rho_{\mathrm{oracle}} - \rho_{\mathrm{base}})$ — the fraction of the baseline-to-oracle gap it closes. Three facts emerge.

First, **the predictor is cheap**. The with-predictions literature usually pictures an expensive learned model; here it is a linear-time count, 0.108 ms, about 2% of the cost of computing OPT once.

Second, **the benefit is real, partial, and never harmful**: a stale forecast reaches 27%–68% gap-capture (falling with staleness) and always stays above the baseline (0.938–0.957 vs 0.923–0.925, even at 30 days).

Third, and this is why: **topology aggregation makes the cheap predictor order-faithful**. The induced degree predictor's order error is only Kendall-$\tau \approx 0.19$–0.32, roughly *half* the raw histogram's (0.38–0.49), and since MPD depends only on order (Chapter 5), the aggregated route survives real drift. Consuming the *same* forecast through the raw histogram is catastrophic: blind FollowPrediction collapses to $0.68 \to 0.36$, far below the ≈ 0.92 baseline — exactly what the robust algorithms of Chapters 4 and 6 are for.

7.2 Six real-world graphs

We re-run the degree-prediction roster of Chapter 4 on the six Network-Repository graphs (two Facebook social, two C. elegans biological, two economic input-output), across the same quality columns, with 95% CIs (**Figure 7.2**). The two load-bearing findings are universal. **F1 holds on all six**: naive MPD fed an adversarial predictor falls below the Ranking floor everywhere — by 0.07

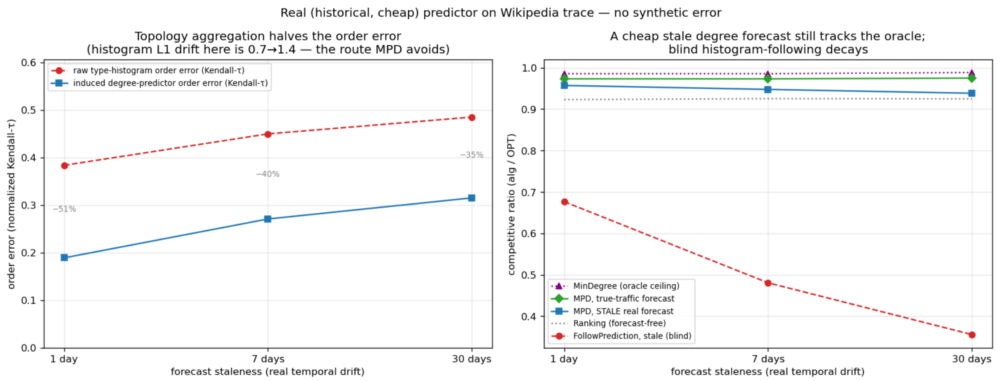


Figure 7.1: A real, cheap predictor (Wikipedia trace): the aggregated degree route survives staleness (b) because aggregation halves the order error (a); blind histogram-following decays.

(Caltech36) to 0.11 (CE-PG). **F3 holds on all six, and confirms its own logic**: the consistency upside is small everywhere (mean +0.049; range +0.022–+0.077) and smallest exactly where the baseline is strongest — the two dense economic graphs, with Ranking already 0.965/0.977, give the tiniest upsides.

The structural-robustness finding **F2 holds qualitatively on all six** (the augmentations are always less sensitive to prediction quality than naive MPD) and *strictly* on the four social/bio graphs (spread 0.22–0.29× MPD's); on the two dense economic graphs the protection is only *partial* — the augmentation cushions the adversarial drop (0.939 vs naive MPD's 0.893) but cannot clear the unusually high 0.965 floor, dipping ≈ 0.03 below it. Those two graphs are so dense that matching is nearly trivial (Ranking ≈ 0.97, MinDegree = 1.00), so there is neither upside to capture (F3) nor much downside to protect: the boundary is F3's own mechanism at work, not an exception to it. Finally, F4 is *dramatic*: the worst-case-designed Feldman/Jaillet–Lu are the weakest advice-free entries on the econ graphs (0.73–0.77) and the augmentation lifts them to 0.99 — a +0.26 rescue.

7.3 Does learning the predictor help?

Because MPD consumes the predictor only through order (Chapter 5), one might train it with a rank loss rather than regression. We tested this and report an honest negative: the advantage is real on deliberately engineered features but *disappears* on real temporal ones, where rank- and regression-trained predictors induce the same order and reach the same ratio. The experiments, their numbers and the three-step argument behind them are in §8.1; what matters for external validity is only the consequence. Once a predictor is order-faithful — which a cheap historical count already is (§7.1) — neither a better algorithm nor a better-trained predictor buys much on average-case matching.

7.4 Chapter summary

On real predictors, real graphs, and a learned predictor, the same wall stands: the advice-free baseline is near-optimal, unguarded following is unsafe, and predictions buy downside protection rather than performance. Having established the wall empirically across every setting we could reach, Chapter 8 reports the directions that tried to get past it, and the concluding outlook (§10.2)

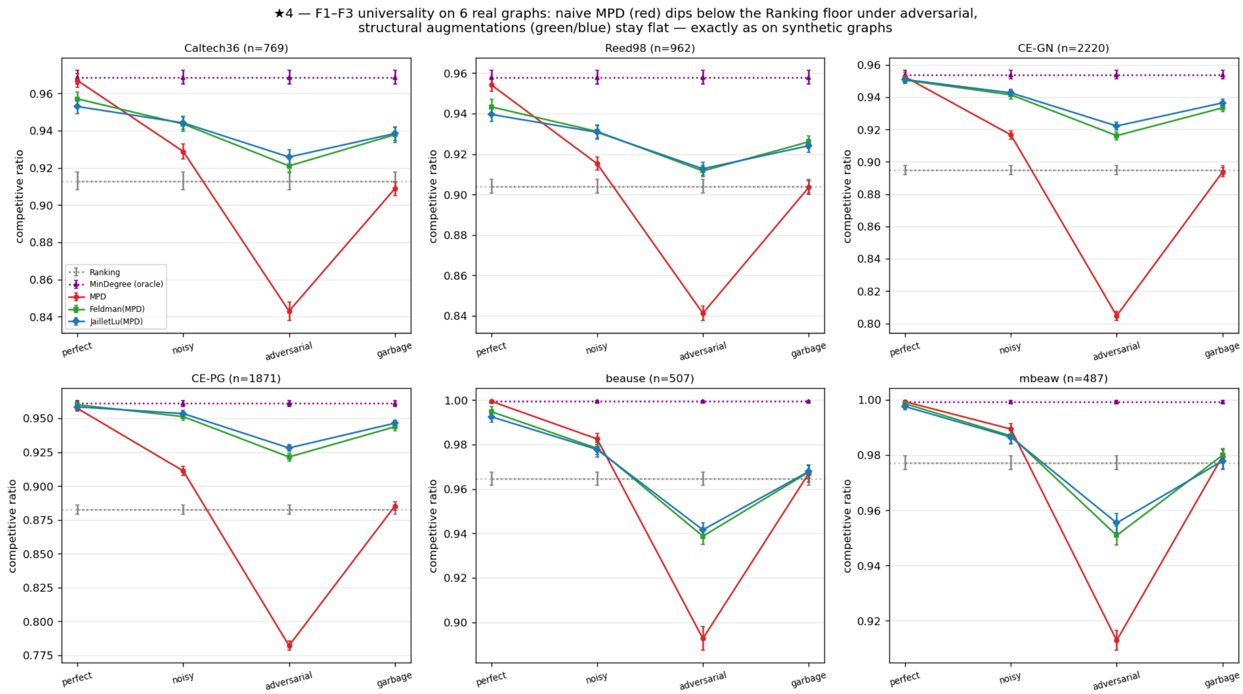


Figure 7.2: F1–F3 on six real graphs: naive MPD (red) dips below the Ranking floor everywhere; the structural augmentations (green/blue) stay flat.

explains why it is forced.

Chapter 8

Exploratory Directions and Negative Results

The experimental chapters (4–7) establish a wall: on average-case matching the advice-free baseline is near-optimal, so predictions are robustness insurance rather than a performance lever. Before accepting the wall as final, we pursued three directions that each *tried to get past it* — learning the predictor to squeeze out more performance (§8.1), finding a serving regime where predictions genuinely help (§8.2), and letting a systematic literature review point us to the highest-value contribution (§8.3). All three returned the same answer, and their convergence is what motivated the theorem. This chapter reports them honestly, including the negatives, because they are part of the evidence and because ruling out ambitious alternatives is itself a result.

8.1 Learning to rank the predictor (a negative result)

Chapter 5 shows that MPD consumes its predictor only through the *order* it induces. This suggests a concrete way to do better: rather than training a predictor to minimize its *regression* error to the true degrees (the standard "predict-then-optimize" objective), train it with an *order-aware* loss — a pairwise rank loss — so it optimizes the quantity the algorithm actually uses. We investigated this in three steps.

M0 — the mechanism exists. With deliberately *divergent* synthetic features (one feature carrying magnitude, another carrying order), a rank-trained linear predictor sharply beats a regression-trained one on the decision metric: matching ratio 0.989 (essentially the oracle) versus 0.974, while the rank-trained predictor has *worse* regression error to the truth (MSE 87.6 vs 33.4) but better order (Kendall-τ 0.058 vs 0.255). The dissociation is real: the worse *fit* gives the better *decision*, confirming that regression is the wrong training objective when it matters.

M1 — but the advantage is doubly gated, and small. Sweeping the feature divergence and the graph difficulty, the rank-advantage is zero when features do not induce an order/magnitude conflict, and zero on easy instances where the baseline is already optimal (the wall, again). It peaks at only +1.3% of the ratio on synthetic graphs; a more favorable framing is *gap-capture*, where rank-training recovers essentially the full oracle gap while regression leaves about 30% of it unrealized — but the gap itself is small.

M3 — and it disappears on real features. The decisive test uses genuine temporal features

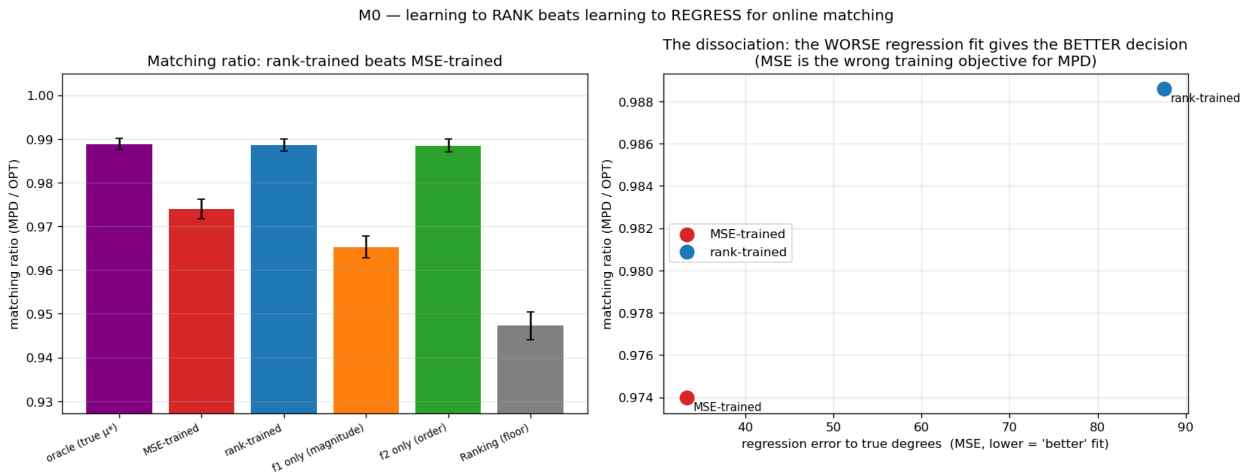


Figure 8.1: M0: with divergent features, rank-training beats regression on the matching ratio (left) despite a worse regression fit (right).

from real serving traces (per-resource reference counts over the previous windows) to predict the next window. Here the rank- and regression-trained predictors produce *identical* order (Kendall-τ 0.126 vs 0.126) and identical matching ratio, across every topology and lag configuration tried. The order/magnitude divergence that powers rank-training is a property of *engineered* features; realistic lagged-count features are co-linear noisy estimates of the same popularity, so regression already recovers the order as well as ranking does.

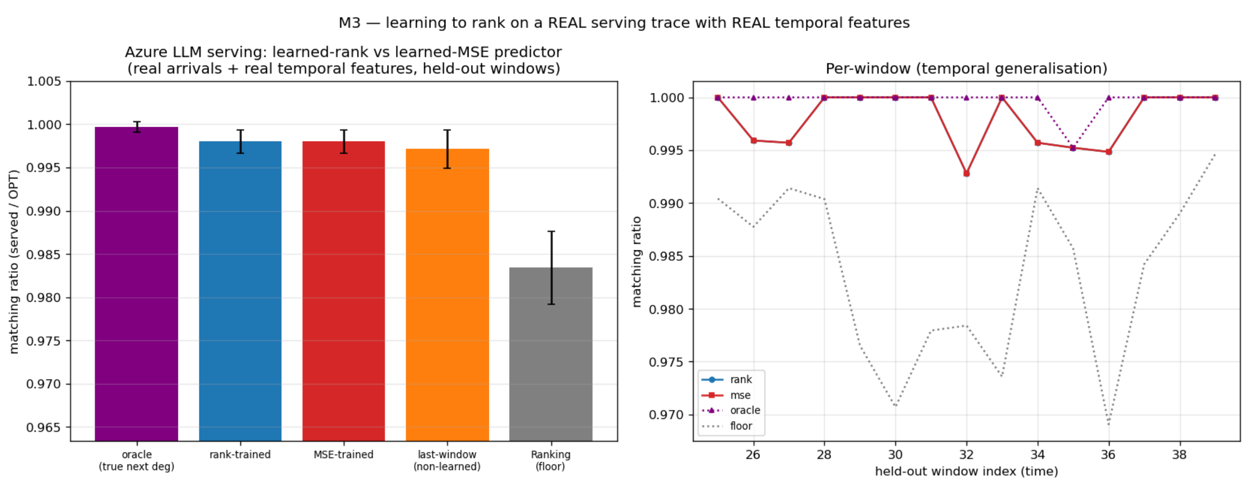


Figure 8.2: M3 (Azure trace, real temporal features): rank- and MSE-trained predictors are indistinguishable — the engineered divergence that powers rank-training does not arise.

Verdict. Learning the predictor with a decision-aligned loss does not elevate to a standalone contribution: the win requires a feature divergence that does not arise in practice, and even where it does the payoff is bounded by the (small) baseline-to-oracle gap. The result is folded into the thesis as a negative that *reinforces* the central finding — once a predictor is order-faithful, which a cheap historical count already is (Chapter 7), neither a better algorithm nor a better-trained predictor buys much on average-case matching.

8.2 Rescuing the serving application with a with-predictions result (a negative probe)

The serving case study (Chapter 9) recovers established systems results and is therefore presented as a case study rather than a novelty claim. We asked whether a *new* actionable with-predictions result could rescue it. The obstacle is again the wall: every serving variant we tried optimizes *throughput* (goodput), and throughput is forgiving — under overload a reactive router fills capacity just as the optimum does, so predictions add nothing. The escape, if one exists, must be a different *objective* on which the reactive baseline is genuinely far from optimal.

We probed the most promising candidate: an **SLO / tail objective** — protecting a tight-SLO class of requests from being dropped — under bursty, non-stationary load, exactly the regime where a reactive policy, lacking foresight, might fail. Using an event-driven simulator we compared non-predictive policies (static capacity reservation; a reactive-adaptive policy that reserves based on *observed* recent load) against a **clairvoyant reference** that reserves based on the *actual future* burst. Two caveats belong with that design. The reference has perfect foresight but is not a proven optimum for the SLO objective, so it *estimates* what foresight is worth rather than bounding it; and the estimate is visibly not tight, because in the moderate regime a trivial static reservation of one slot drives tight-SLO violations to near zero and *beats* the clairvoyant reference outright — reserving for the actual future burst over-reserves there. What the sweep does establish is one-directional and still useful: across every regime — overload level, uniform vs bursty tight-SLO demand — the best non-predictive policy comes within $\leq 3\%$ of a policy that knows the future exactly, so the particular thing a forecast would supply is not what these policies are missing. Two reasons, both robust to the sweep: protecting a tight-SLO minority needs only a small static headroom, no forecast; and bursts are persistent enough that reacting to observed load is almost as good as forecasting it.

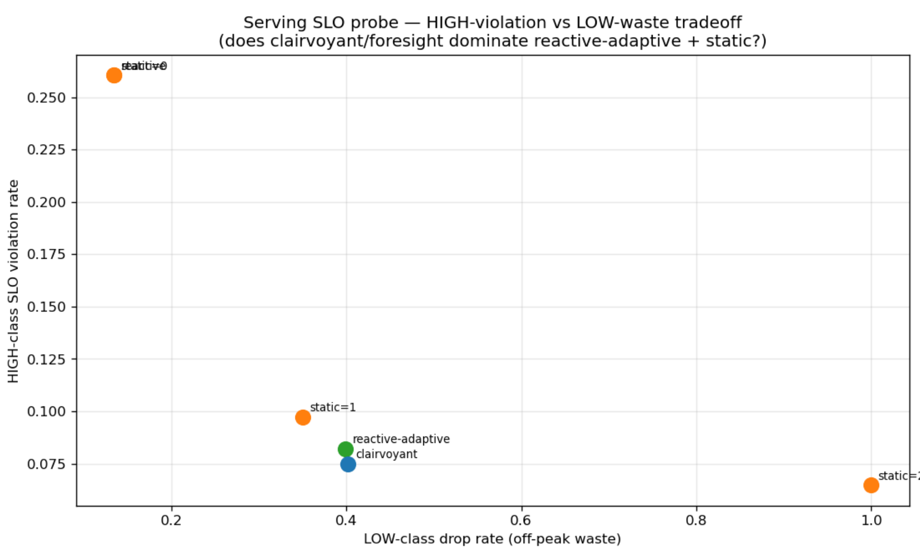


Figure 8.3: Serving SLO probe: a non-predictive policy matches the clairvoyant oracle to within a few percent — foresight does not help.

Verdict. The tail objective is forgiving too — a third face of the wall, after throughput (Chapters 4–7) and predictor-learning (§8.1). We found no natural regime where foresight helps, so serving remains a case study. A regime that would break the wall (a non-stationary or adversarial objective where the baseline is far from optimal) is exactly the kind of setting the thesis brackets as future work.

8.3 A literature review that redirected the work

Deciding *which* of our findings were genuinely novel — and worth developing — required a systematic prior-art review rather than intuition. We searched the prior art from several independent starting points and checked each candidate-novelty claim against the primary papers rather than against their abstracts. The review returned honest, sometimes deflating, verdicts. It confirmed that the unified experimental benchmark and the empirical study of test-and-fallback are unoccupied and worth leading with; that the order-error finding is *partially* pre-empted by ACI's Appendix D and must be reframed as a tightness/measure characterization rather than a discovery (Chapter 5); and that the serving results are largely re-derivations of established systems facts, warranting their demotion to a case study (§8.2, Chapter 9). A second, focused prior-art pass on the theory question confirmed that no prior work quantifies the cost of the prefix test on strong-baseline instances, while flagging the one risk any such result must defend against (Choo et al.'s constructive baseline-coupling) — both inputs to the outlook of §10.2 and to the companion theoretical work it defers to.

This review is itself part of the research process reported here: it turned an undifferentiated pile of findings into a prioritized contribution, redirected effort away from low-value elaborations (weighted-value variants, more baseline comparisons) and toward the theory question, and enforced the honesty guardrails carried throughout the thesis.

8.4 Synthesis: the negatives point to the same wall

The three explorations point the same way. A better-trained predictor does not help on real features (§8.1); a with-predictions lens does not rescue serving even on a tail objective (§8.2); and the literature review confirmed there was no easy performance win to be had (§8.3). Together with the throughput wall of Chapters 4–7, they show the phenomenon is not confined to one objective, one algorithm, or one dataset — it recurs everywhere we pushed. That robustness of the empirical finding is precisely what suggested it might be *forced* rather than accidental — the question the concluding outlook (§10.2) takes up, and which the companion theoretical work in preparation pursues in full.

Chapter 9

Application Case Study: AI-Inference Serving

The thesis's abstraction is not confined to classical matching markets; it instantiates a contemporary systems problem. This chapter casts request routing in an AI-inference serving system as online matching. It does two things. It tests whether the abstraction is rich enough to carry a live systems problem — the answer is that it recovers the field's established results, which is the strongest evidence available that the mapping is faithful — and it supplies the setting in which Chapter 8 probed, and failed to find, a regime where predictions genuinely help. Because the systems facts recovered below are already known, the chapter is a **case study rather than a novelty claim**; that decision follows the prior-art review of Chapter 8.

9.1 The serving instantiation

We map serving to online **capacitated matching**: each offline resource is a model replica or cache shard that can serve up to c concurrent requests, so we match with capacity c rather than 1 (this is b-matching with every b equal to c; we write c throughout, including in the figures). Arrivals are requests drawn from a non-uniform, bursty traffic distribution over request types, and an edge is a capability or cache affinity. Two quantities must be kept apart. **Goodput** is the fraction of arriving requests that are served; the **competitive ratio** is the number served divided by the capacitated optimum on the same realized instance. They coincide only when the optimum can serve every arrival, which under overload it cannot. Both appear below and each figure's axis states which it plots: Figure 9.1 normalizes by the optimum (*served / best possible*), Figure 9.2 by the arrivals (*served / total*), and Figure 9.3 reports a different quantity again, the KV-cache hit fraction. We instantiate this on three real traces (Wikipedia pageviews, an Azure LLM inference trace, and the Mooncake prefix-cache trace [20]) and across four serving concerns.

Capacity as robustness (**Figure 9.1**). Increasing the capacity c smooths the effect of a bad prediction: a capacity-aware baseline stays safe, while *blindly* trusting a traffic forecast degrades *further* as capacity grows. Capacity is thus a *substitute* for algorithmic robustness — the systems analogue of the robustness-insurance thesis.

Forecasts vs live load (**Figure 9.2**). Under dynamic service times (requests hold a slot for a real duration, released event-by-event), a live-load signal beats a stale traffic forecast — a reactive

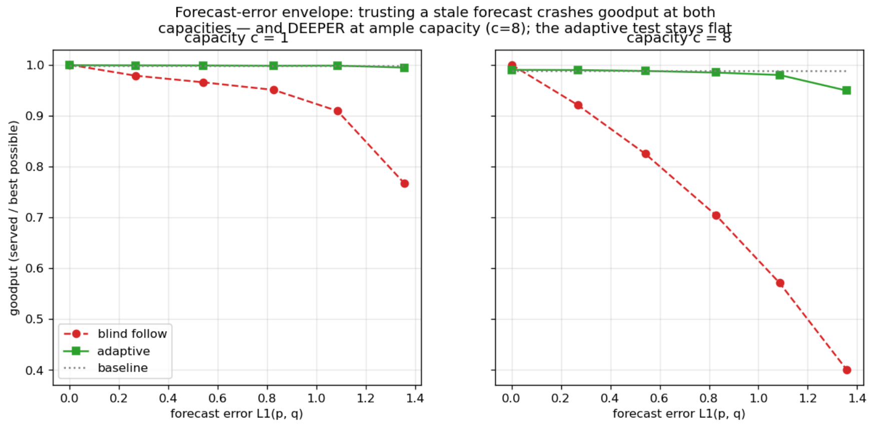


Figure 9.1: Capacity as robustness: blindly following the forecast crashes goodput — deeper at ample capacity ($c = 8$) — while the adaptive test stays flat.

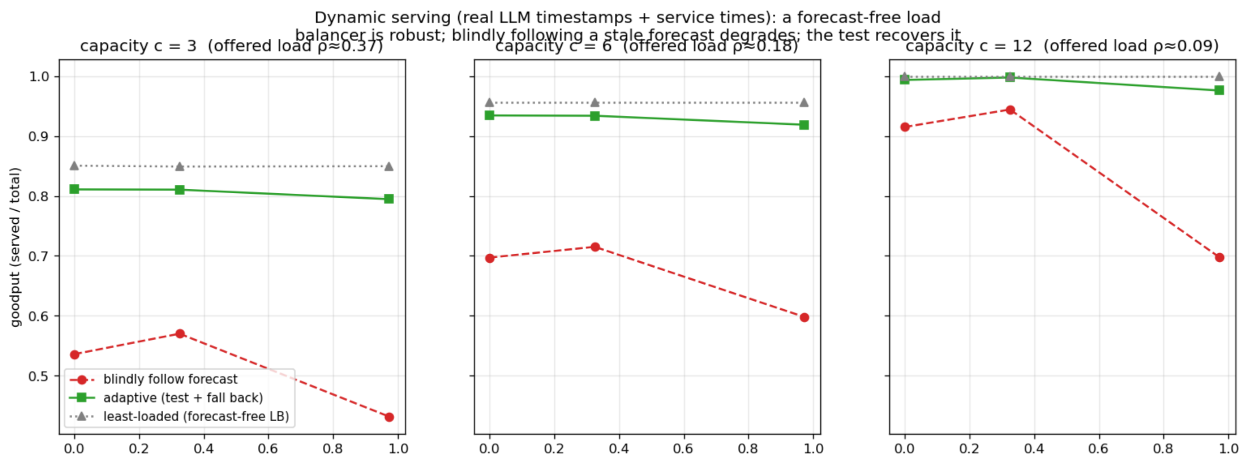


Figure 9.2: Live load beats a stale forecast under dynamic service times; an adaptive test recovers most of the gap.

load balancer outperforms a forecast-following router when the forecast has aged.

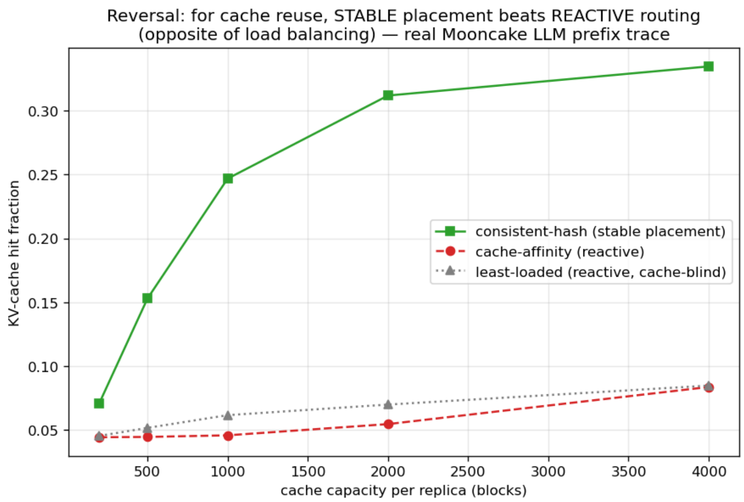


Figure 9.3: The cache-affinity reversal (Mooncake trace): stable placement beats reactive routing for KV-cache reuse.

Cache-affinity routing (**Figure 9.3**). For prefix-cache-aware routing, a *stable* affinity router beats a reactive one — the reverse of the traffic-forecast case, because cache locality rewards persistence [21].

Each of these recovers an established systems result cleanly, demonstrating that the framework instantiates the problem faithfully.

9.2 A probe for a genuinely new result, and its negative

We also asked whether the with-predictions lens yields a *new* actionable serving result on a tail rather than a throughput objective. It does not: on an SLO objective — protecting a tight-SLO class of requests under bursty load — a non-predictive policy comes within $\leq 3\%$ of one with perfect foresight across every regime swept. The experiment, its caveats and its two explanations are in §8.2. The consequence for this chapter is that we found no natural regime where foresight helps, so serving remains a case study.

9.3 Chapter summary

The serving instantiation shows the framework reaches a live systems problem and recovers its established results — capacity as a robustness substitute, live load over stale forecasts, stability for cache locality — and that even a tail objective does not open a genuine with-predictions win.

Chapter 10

Conclusion and Future Work

10.1 Summary

This thesis studied learning-augmented algorithms for online bipartite matching by first building a common experimental foundation and then explaining what it revealed. On a single harness — validated against the published Borodin et al. study (Chapter 3) — we compared the advice-free baselines, MinPredictedDegree and its augmentations, and the test-and-fallback algorithms, across synthetic graphs, six real-world graphs, and real request traces (Chapters 4–7). Across every setting the same picture appeared: the advice-free baseline is already near-optimal on average-case inputs, so the *consistency* upside of a good prediction is small; unguarded prediction-following crashes *below* the baseline under bad predictions; and the value of the sophisticated algorithms is downside protection, delivered by either a structural or an adaptive robustness mechanism with distinct trade-offs. We sharpened the sub-questions — that the (small) loss is governed by a Kendall-τ order error rather than magnitude (Chapter 5, crediting the prior order-dependence result of ACI), and that the adaptive test has a threshold-calibration pathology and a resolution limit (Chapter 6) — and we honestly reported the directions that did *not* pan out: learning the predictor for extra performance, and finding a serving regime where predictions genuinely help (Chapter 8).

The recurring wall raises an obvious question: is it an accident of the inputs we chose, or is it forced? The next section gives our answer in outlook form. The single sentence that unifies the thesis is: **on average-case online matching, predictions are robustness insurance rather than a performance lever — and the upside they offer is smaller than the price of finding out whether to trust them.**

10.2 A theoretical outlook: the price of testing advice

This section does two things and nothing more: it proves one inequality, and then reads our own experiments through it. Nothing beyond that inequality is claimed as a theorem, and the reading that follows it is an interpretation of measurements rather than a proof.

With the scope fixed: the experiments say that no *practical* acceptance threshold captures the upside safely (§6.3, Figure 6.4). The question here is whether a decision rule of some *other* kind could, on the inputs where the wall stands.

A trade-off inequality. Let G and Bd be two instance distributions sharing the *same* advice, such that following the advice gains δ under G and loses Δ under Bd relative to the advice-free baseline, and let γ_k be the total-variation distance between the laws of their length-k prefixes. Then every test-and-fallback algorithm — deciding by *any* measurable rule on its prefix — satisfies

$$(1-\eta_c) \;\leq\; \eta_r + \gamma_k + o(1),$$

where η_c is the fraction of the upside it forgoes under G and η_r its robustness loss under Bd as a fraction of Δ.

The proof is a short conditioning argument: capturing the upside forces the algorithm to follow with high probability under G; robustness forces fallback under Bd; and no function of the prefix can behave differently on two prefix distributions that are statistically γ_k-close. The inequality thus converts "consistent *and* robust" into a question about *sample complexity*: how long must the prefix be before it distinguishes advice worth following from advice worth rejecting?

The reading applies to the rare-resource instances that produce Figure 6.4 — a *decomposable* family, in which the instance splits into independent cells and the advice acts on each of them separately — and it is a **budget–stakes** relation. Write δ for the *stakes*: what good advice gains over the baseline. In the companion development, which is not proved here, the decision costs a prefix of $k^* = \tilde{\Theta}(\theta/\delta^2)$ — the inverse square of the stakes, scaled by a contention parameter θ that on these instances is of the same order as the baseline slack $1-\rho_{\text{base}}$ — and that budget is met by a simple *directional* statistic (do the prefix arrivals agree with the advice's predictions more often than they contradict them?). The stakes are in turn capped by the same slack, $\delta = O(1-\rho_{\text{base}})$: advice can only win what the baseline leaves on the table. Substituting, the reading predicts that on strong-baseline instances the required prefix exceeds the whole instance — that an upside below $\Theta(\sqrt{(1-\rho_{\text{base}})/n})$ would not be captured by any rule at any prefix length $k \leq n$. At the parameters of Chapter 4 ($\rho_{\text{base}} \approx 0.99$, $n = 2000$) that threshold is ≈ 0.004, and the upsides we measured are of exactly this order (F3): the empirical wall sits where the reading says it should. Whether the same relation holds beyond decomposable families is open (§10.4). This is Figure 6.4 read quantitatively: the potential upside and the capturable upside separate as the baseline strengthens, because the stakes shrink faster than the test's resolution improves.

Two honest notes bound the scope of this outlook. First, the budget law cuts both ways: where the stakes are large (a weak baseline), the directional statistic is cheap and following good advice is easy — the wall is a statement about strong-baseline, average-case inputs, not about testing in general. In particular, a tempting stronger conjecture — that the near-linear sample cost of *tolerant distribution testing* (§2.5) blocks every sublinear rule outright — is false: it is refuted by the same directional statistic, and the ℓ_1-threshold blindness of §6.3 is a property of testing the *distance* rather than the *payoff*. Second, this thesis claims here only the inequality displayed above and the quantitative reading of its own experiments; the two-sided law, its proofs, and its exact scope are the subject of the companion work.

10.3 Limitations

- **Input model.** We work in the known-i.i.d. model. Because Known-I.I.D. $\leq$ Random-Order in difficulty, the algorithms' guarantees carry over, but the empirical wall is an *average-case* statement; we do not claim it for adversarial arrival order.

- **Test model.** Following the original authors, the test-and-fallback experiments use an empirical-ℓ_1 surrogate for the (unimplemented) distribution tester; §10.2 explains why the surrogate's blindness is structural rather than an implementation artifact.
- **Prediction-object heterogeneity.** The degree- and histogram-prediction families do not map onto every graph, which is why they are reported in parallel panels rather than one table.
- **Data breadth.** Each real modality is exercised by one trace.
- **Objective.** We evaluate matching size (goodput) only. On objectives where the advice-free baseline is *not* near-optimal — tail latency, per-type fairness, migration or recompute cost — the picture could differ; our one probe in that direction (§8.2) closed the simplest such attempt but not the space (§10.4).
- **Theory scope.** The thesis deliberately confines its theory to the outlook of §10.2 — one proved trade-off inequality and a quantitative reading of the experiments. The full budget–stakes law is companion work in preparation, and no theorem beyond the stated inequality is claimed here.

10.4 Future work

The thesis brackets, rather than resolves, the settings in which predictions might genuinely help online matching, and these are the natural next directions.

- **Beyond average-case inputs.** The wall is an average-case phenomenon. Adversarial or non-stationary arrival orders, where the advice-free baseline is provably far from optimal, are where predictions should carry real value — and where a *positive* counterpart to this thesis's wall might be proved.
- **Beyond throughput.** Objectives on which the baseline is not near-optimal — tail latency, per-type fairness, migration or recompute cost — may admit genuine with-predictions gains that the goodput objective forecloses; our serving SLO probe (Chapter 8) closed the simplest such attempt but not the space.
- **Completing the theory.** Finishing the companion budget–stakes development — the sharp two-sided law, the directional statistic as its matching upper bound, and its exact scope (decomposable families; whether a non-decomposable family can push the budget higher is open) — would turn the outlook of §10.2 into a tight characterization.
- **Better tests, honestly.** Whether a super-linear or amortized test, or a test that reuses online decisions as samples, can beat the stakes-squared budget of §10.2 is an open and practically motivated question.

10.5 Closing

Predictions are a powerful tool for online algorithms, but this thesis is a study of their *limits* on one well-understood problem. On average-case online matching, the honest verdict is that a cheap, order-faithful predictor already captures nearly all there is to capture, that the sophisticated machinery earns its keep as insurance rather than as performance, and that finding out whether to trust a prediction costs more, on these inputs, than the prediction is worth. Recognizing where predictions cannot help is, we hope, as useful as knowing where they can.

References

[1] R. M. Karp, U. V. Vazirani, and V. V. Vazirani. An optimal algorithm for on-line bipartite matching. *STOC*, 1990.

[2] J. Feldman, A. Mehta, V. Mirrokni, and S. Muthukrishnan. Online stochastic matching: Beating 1-1/e. *FOCS*, 2009.

[3] V. H. Manshadi, S. Oveis Gharan, and A. Saberi. Online stochastic matching: Online actions based on offline statistics. *Mathematics of Operations Research*, 37 (4), pp. 559–573, 2012.

[4] P. Jaillet and X. Lu. Online stochastic matching: New algorithms with better bounds. *Mathematics of Operations Research*, 39 (3), pp. 624–646, 2014.

[5] A. Borodin, C. Karavasilis, and D. Pankratov. An experimental study of algorithms for online bipartite matching. *ACM Journal of Experimental Algorithmics*, 25, 2020.

[6] M. Mitzenmacher and S. Vassilvitskii. Algorithms with predictions. *Beyond the worst-case analysis of algorithms*, Cambridge University Press, pp. 646–662, 2020.

[7] T. Lykouris and S. Vassilvitskii. Competitive caching with machine learned advice. *ICML*, 2018.

[8] A. Wei and F. Zhang. Optimal robustness-consistency trade-offs for learning-augmented online algorithms. *NeurIPS*, 2020.

[9] J. Chłędowski, A. Polak, B. Szabucki, and K. Żołna. Robust learning-augmented caching: An experimental study. *ICML*, 2021.

[10] T. Yoshinaga and Y. Kawase. Analyzing the effect of prediction accuracy on the distributionally-robust competitive ratio. *arXiv preprint arXiv:2601.06813*, 2026.

[11] A. Aamand, J. Y. Chen, and P. Indyk. (Optimal) online bipartite matching with degree information. *NeurIPS*, 2022.

[12] D. Choo, T. Gouleakis, C. K. Ling, and A. Bhattacharyya. Online bipartite matching with imperfect advice. *Proceedings of the 41st international conference on machine learning (ICML)*, 2024.

[13] J. Jiao, Y. Han, and T. Weissman. Minimax estimation of the L_1 distance. *IEEE Transactions on Information Theory*, 64 (10), pp. 6672–6706, 2018.

[14] K. Burathep, T. Erlebach, and W. K. Moses Jr. Learning-augmented online bipartite matching in the random arrival order model. *Proceedings of the 51st international conference on current trends in theory and practice of computer science (SOFSEM)*, 2026.

[15] L. Paninski. A coincidence-based test for uniformity given very sparsely sampled discrete data. *IEEE Transactions on Information Theory*, 54 (10), pp. 4750–4755, 2008.

[16] G. Valiant and P. Valiant. An automatic inequality prover and instance optimal identity testing. *SIAM Journal on Computing*, 46 (1), pp. 429–455, 2017.

[17] G. Valiant and P. Valiant. Estimating the unseen: An $n/\log n$-sample estimator for entropy and support size, shown optimal via new CLTs. *STOC*, 2011.

[18] C. L. Canonne, A. Jain, G. Kamath, and J. Li. The price of tolerance in distribution testing. *COLT*, 2022.

[19] J. E. Hopcroft and R. M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 1973.

[20] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *arXiv preprint arXiv:2407.00079*, 2024.

[21] V. Srivatsa, Z. He, R. Abhyankar, D. Li, and Y. Zhang. Preble: Efficient distributed prompt scheduling for LLM serving. *arXiv preprint arXiv:2407.00023*, 2024.

Appendix A

Reproduction Guide

Every quantitative result in this thesis is regenerated from a fixed seed by a single script. Two classes of script must be distinguished. Most are *self-contained*: they generate their own synthetic instances and run as-is on a clean checkout. The rest — the real-graph and real-trace results of Chapters 7, 8 and 9 — first need external data placed under `data/`; that data is not redistributed with the code, and §A.5 records what each dataset is and where it comes from. In the map below, a dagger (†) marks the scripts of the second class.

This appendix maps each figure and table to its script, gives the commands and runtimes, lists the full Phase-2 reproduction tables deferred from Chapter 3, and records data provenance.

A.1 Environment and principles

- **Stack:** Python 3.12; NumPy 1.26+, SciPy 1.13, NetworkX 3.3, Matplotlib (plots only).
- **Determinism:** all randomness derives from one master seed (default `0`) via NumPy's `default_rng(seed).spawn(k)`, which yields independent, reproducible streams (graph, instance, algorithm randomness, prediction perturbation). Results are bit-stable across NumPy minor versions from 1.25 (BitGenerator-based spawning).
- **Paired trials:** within a comparison, every algorithm and error level reuses the same graph, arrival sequence, `OPT`, and tie-break seed, so differences are attributable to the prediction alone.
- **Confidence intervals:** 95% normal-approximation half-widths over trials.
- Each script writes `results/<name>.json` (raw means/CIs) and `results/<name>.png` (plot), and prints per-step progress.

A.2 Figure / table → script map

Thesis object	Script	Output	≈ runtime
Fig 3.1 (ER U-curve)	`scripts/run_er_full.py`	`results/er_full.{json,png}`	~20 min
Fig 3.2 (left-regular)	`scripts/run_left_regular.py`	`results/left_regular.{json,png}`	~10 min
Table 4.1 (unified benchmark)	`scripts/run_unified_benchmark.py`, then `plot_unified_panels.py`	`results/unified_benchmark.{json,png}`, `unified_benchmark_panel{A,B,C}.png`, `unified_benchmark_tables.md`	~100 s

Thesis object	Script	Output	≈ runtime
Fig 4.1 (consistency–robustness plane)	`scripts/run_consistency_robustness.py`	`results/consistency_robustness.{json,png}`	~1 s
Fig 5.1 (order-error vs ACI)	`scripts/run_order_vs_theory.py`	`results/order_vs_theory.{json,png}`	~30 s
Fig 6.1 (envelope), Fig 6.2 (prefix sweep)	`scripts/run_choo_bem.py`	`results/choo_bem_{envelope,prefix}.png`	~20 min
Fig 6.3, recalibration (§6.3)	`scripts/run_recalibration.py`	`results/recalibration_*.png`	~1.5 min
Fig 6.4 (testing-wall frontier)	`scripts/run_impossibility_frontier.py`	`results/impossibility_frontier.{json,png}`	~6 s
Fig 7.1 (real predictor) †	`scripts/run_real_predictor.py`	`results/real_predictor.{json,png}`	~15 s
Fig 7.2 (six real graphs) †	`scripts/run_realworld_robustness.py`	`results/realworld_robustness.{json,png}`	~65 s
Fig 8.1 (M0 rank vs MSE)	`scripts/run_rank_vs_mse_mve.py`	`results/rank_vs_mse_mve.{json,png}`	~10 s
M1 sweep (§8.1, no figure)	`scripts/run_rank_when_it_matters.py`	`results/rank_when_it_matters.{json,png}`	~20 s
Fig 8.2 (M3 real-trace learning) †	`scripts/run_rank_real_trace.py`	`results/rank_real_trace.{json,png}`	~10 s
Fig 8.3 (serving SLO probe)	`scripts/run_serving_slo_probe.py`	`results/serving_slo_probe.{json,png}`	~1 s
Figs 9.1–9.3, serving (Ch 9) †	`scripts/run_serving.py`, `run_serving_trace.py`, `run_serving_dynamic.py`, `run_prefix_cache.py`	`results/serving_*.png`, `prefix_cache_*.png`	varies
Outlook checks (§A.6)	`scripts/verify_witness_gap.py`	console output	seconds
Real-world Borodin Tables 3/4 (validation) †	`scripts/run_realworld.py`	`results/realworld.json`	~few min

A.3 Reproduction commands

```
# from the project root (matching-experiments/)
# correctness anchors - 7 hand-verifiable test files, all pass:
for t in tests/test_*.py; do python3 "$t"; done

# regenerate the headline results (fast ones):
python3 scripts/run_unified_benchmark.py        # Table 4.1 (data)
python3 scripts/plot_unified_panels.py          # Table 4.1 (panel charts)
python3 scripts/run_order_vs_theory.py          # Fig 5.1
python3 scripts/run_real_predictor.py           # Fig 7.1
python3 scripts/run_realworld_robustness.py     # Fig 7.2
python3 scripts/run_impossibility_frontier.py   # Fig 6.4
python3 scripts/run_rank_vs_mse_mve.py          # Fig 8.1
```

```
python3 scripts/run_rank_when_it_matters.py      # M1 (§8.1)
python3 scripts/run_rank_real_trace.py           # Fig 8.2
python3 scripts/run_serving_slo_probe.py         # Fig 8.3
python3 scripts/run_consistency_robustness.py    # Fig 4.1 (replots Table 4.1)

# the long (max-flow / Hopcroft-Karp-bound) sweeps:
python3 scripts/run_er_full.py                   # Fig 3.1  (~20 min)
python3 scripts/run_left_regular.py              # Fig 3.2  (~10 min)
python3 scripts/run_choo_bem.py                  # Fig 6.1, 6.2 (~20 min)
```

All scripts hardcode seed `0` unless noted; re-running reproduces the reported numbers.

A.4 Full Phase-2 reproduction tables (deferred from §3.6)

Setup: $n = 1000$, $m = n$, 100 trials per parameter value, seed 0. Target is *qualitative* agreement with Borodin et al. (Python/NetworkX vs the paper's C++/Edmonds–Karp; absolute differences ≤ 0.02 accepted).

Erdős–Rényi, competitive ratio at selected c (SG=SimpleGreedy, Rk=Ranking, F/J = Feldman/Jaillet–Lu, -NG/-G = non-greedy/greedy):

c	SG	Rk	F-NG	F-G	J-NG	J-G
0.10	0.9995	0.9994	1.0000	1.0000	0.9991	1.0000
1.90	0.9362	0.9363	0.9033	0.9637	0.9202	0.9602
4.90	**0.8640**	0.8655	0.7648	**0.8845**	0.7949	**0.8854**
8.90	0.9094	0.9088	0.7314	0.9123	0.7662	0.9120
14.90	0.9487	0.9486	0.7290	0.9488	0.7640	0.9483

Per-algorithm minima (ER): SG 0.8640 @ c=4.9; Rk 0.8649 @ 4.7; F-NG 0.7288 @ 13.5; F-G 0.8833 @ 5.3; J-NG 0.7616 @ 14.1; J-G 0.8839 @ 5.3.

Random left-regular, competitive ratio at selected d:

d	SG	Rk	F-NG	F-G	J-NG	J-G
1	1.0000	1.0000	0.9845	1.0000	0.9845	1.0000
2	0.9539	0.9537	0.8769	0.9679	0.8945	0.9659
5	**0.8905**	0.8900	0.7595	0.9008	0.7909	0.9022
10	0.9275	0.9278	0.7317	0.9276	0.7677	0.9290
30	0.9770	0.9767	0.7308	0.9757	0.7633	0.9754

Paper-claim checklist (all verified ✓): greedy minima near $c \approx 4.9$ / $d = 5$; Ranking $\approx$ SimpleGreedy (max diff 0.0017 ER, 0.005 LR — the paper omits Ranking's curve for this reason); non-greedy variants degrade monotonically as c, d grow; greedy complex variants $\approx$ SimpleGreedy asymptotically; the c=14.9 non-greedy ordering (J-NG 0.764 $>$ F-NG 0.729) matches the paper's worst-case-bound ordering. Across families, the non-greedy algorithms converge to the same asymptotic constants (0.729 Feldman, 0.764 Jaillet–Lu, within 0.002), above their worst-case bounds by +0.06 / +0.03; §3.6 reads that observation.

A.5 Data provenance

Real data is stored locally under `data/` (large; excluded from version control).

- **Real graphs (Chapter 7, §3.6 validation):** six graphs from the Network Repository (`networkrepository.com`) — `socfb-Caltech36`, `socfb-Reed98`, `bio-CE-GN`, `bio-CE-PG`, `econ-beause`, `econ-mbeaflw` — downloaded as MatrixMarket `.mtx` / whitespace `.edges`, reduced to simple undirected graphs and converted to bipartite by random balanced partition (Borodin Table 3) or duplicating double-cover (Table 4).
- **Traces (Chapters 7, 8, 9):** Wikipedia "top articles per day" for four days, retrieved from the Wikimedia REST pageviews API (`data/trace/wiki/`, used as live day vs 1/7/30-day-stale forecasts); the Azure LLM inference trace from Microsoft's public Azure dataset release (`data/trace/azure_llm/`, context/generated token counts with timestamps); and the Mooncake conversation trace released with [20] (`data/trace/mooncake/`, per-request `hash_ids` for prefix-cache blocks). The Wikipedia trace is a snapshot of a live source rather than a static benchmark, so exact reproduction needs the same snapshot, not merely the same API.

A.6 Outlook verification snippets (§10.2)

Every quantity quoted in the outlook of §10.2 was checked numerically before being used. There are three checks, each a short simulation of the rare-resource construction.

1. **Per-cell constants.** The per-cell optimum, the per-cell baseline, and the closed forms for the advantage and for ℓ_1 agree with simulation to three decimals. For example, at contention $\theta = 0.6$ and advice bias 0.3 the simulated per-cell advantage is ± 0.119, against a predicted $\pm\theta \cdot 0.3 = \pm 0.12$.
2. **The conversion law.** The expected ratio when the advice is followed matches $\rho_{\text{perfect}} - \frac{1}{2}\ell_1(p, q)$ — it falls off linearly in the advice's ℓ_1 error — again to three decimals. This is what makes the stakes δ of §10.2 a linear function of the advice error rather than an arbitrary quantity.
3. **The two budget–stakes ingredients.** The directional statistic's accuracy at a fixed prefix length does not degrade as n grows, and the plug-in ℓ_1 estimate becomes blind — indistinguishable between good and bad advice — once $k \ll r$. Both are produced by `scripts/verify_witness_gap.py` (§A.2).

These checks are what license the *reading* of §10.2; they are not a proof of it. The formal development is separate work outside this thesis.